

Final Product Specification — X Protocol Research + Production Ingestion Platform

Status: Canonical product-level specification synthesized from the active documentation and research in:

- ahansardar/x-scraper
- techtiesai-png/X-rev-os
- techtiesai-png/XINGESTIONV2

This document explains the **complete intended product**. It does not replace repository-specific implementation ledgers or detailed subsystem architecture documents. Where an older idea conflicts with the active `X-rev-os` or `XINGESTIONV2` standing documents, the newer active architecture wins.

The product must only operate with accounts, sessions, data, and network access the operator is authorized to use. It must not include credential theft, account takeover, CAPTCHA/challenge bypass, or access-control circumvention.

1. Executive Summary

The final product is **not merely an X/Twitter scraper**.

It is a complete, evidence-driven ingestion platform with two major active subsystems:

1. X-rev-os

- researches and understands X's observable web protocol;
- captures authorized browser/network evidence;
- identifies operation/request behavior;
- builds first-party request, parsing, and pagination logic;
- validates exact protocol compositions;
- exports versioned, approved protocol releases.

2. XINGESTIONV2

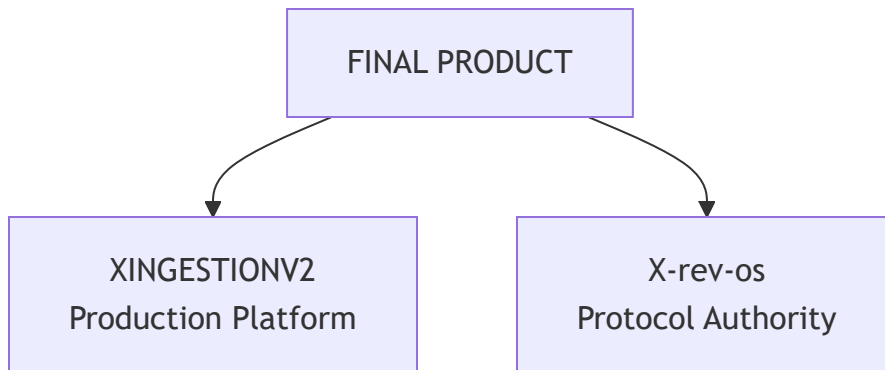
- exposes stable product capabilities;

- manages durable tasks and workers;
- owns all production retries;
- manages accounts, sessions, and network allocation;
- persists raw production evidence;
- normalizes data into canonical models;
- handles monitoring, analytics, APIs, and parent-system integration;
- pins and deploys approved X-rev releases.

The original `x-scraper` repository remains useful as the **historical research/prototype origin**. Many of its concepts survive, but several have been formalized into stronger abstractions in X-rev-os and XINGESTIONV2.

2. Why the Product Is Split into Two Active Repositories

The two-repository structure is intentional.



The two systems change at different speeds.

X protocol behavior may change frequently:

- operation IDs;
- feature bundles;
- request metadata;
- response variants;
- pagination behavior;
- auth attachment semantics;
- client-side transaction metadata.

The production control plane should not need to change every time one of those protocol details changes.

Likewise, protocol researchers should not need to touch:

- PostgreSQL task state;
- Redis delivery;
- retry generations;
- session leases;
- analytics;
- APIs;
- production data models.

Therefore:

X-rev-os owns X-specific protocol behavior. XINGESTIONV2 owns production ingestion and orchestration.

There must be only one canonical implementation of X-specific request construction, parsing, pagination, and related protocol logic.

3. Historical Role of x-scraper

x-scraper established the original research direction.

Its important ideas include:

- use authenticated structured web/GraphQL responses rather than DOM scraping for the primary data path;
- isolate endpoint-specific logic;
- manage authenticated sessions;
- detect authentication failures separately from endpoint/schema failures;
- classify failures;
- normalize raw responses;
- isolate pagination logic;
- retain debug/raw responses;
- monitor request/response changes;
- use regression fixtures and contract tests;

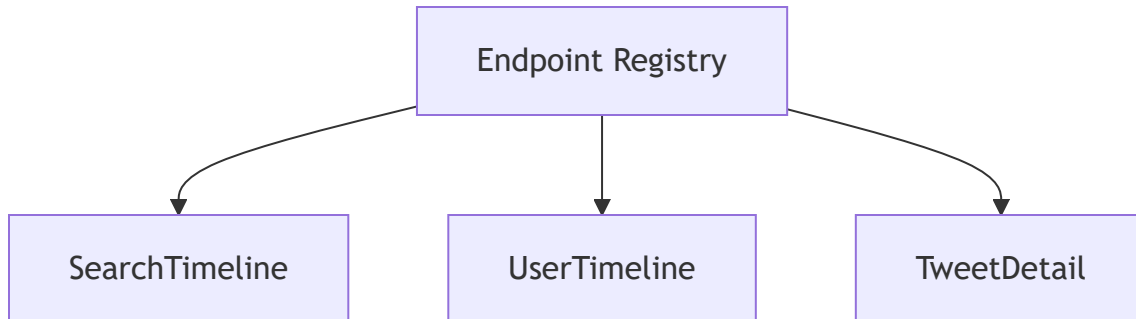
- build observability around the collection layer.

These ideas remain valid.

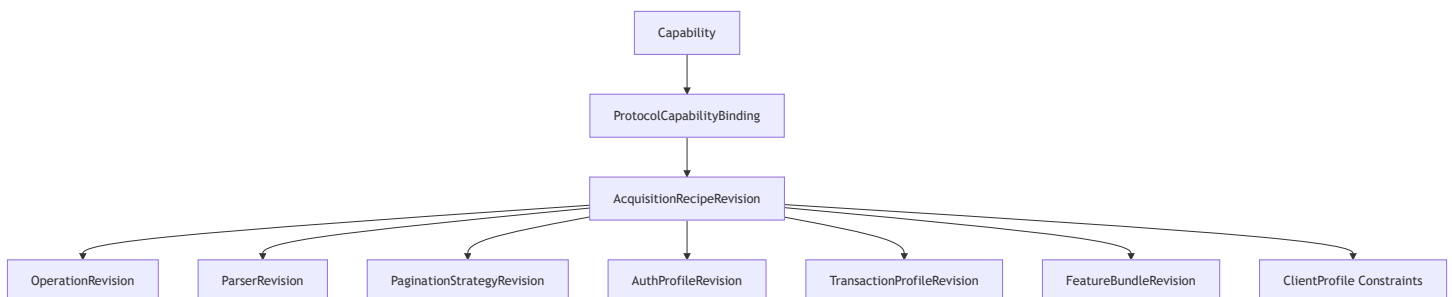
However, some original abstractions are superseded.

3.1 Superseded: simple Endpoint Registry

Old concept:



Final concept:

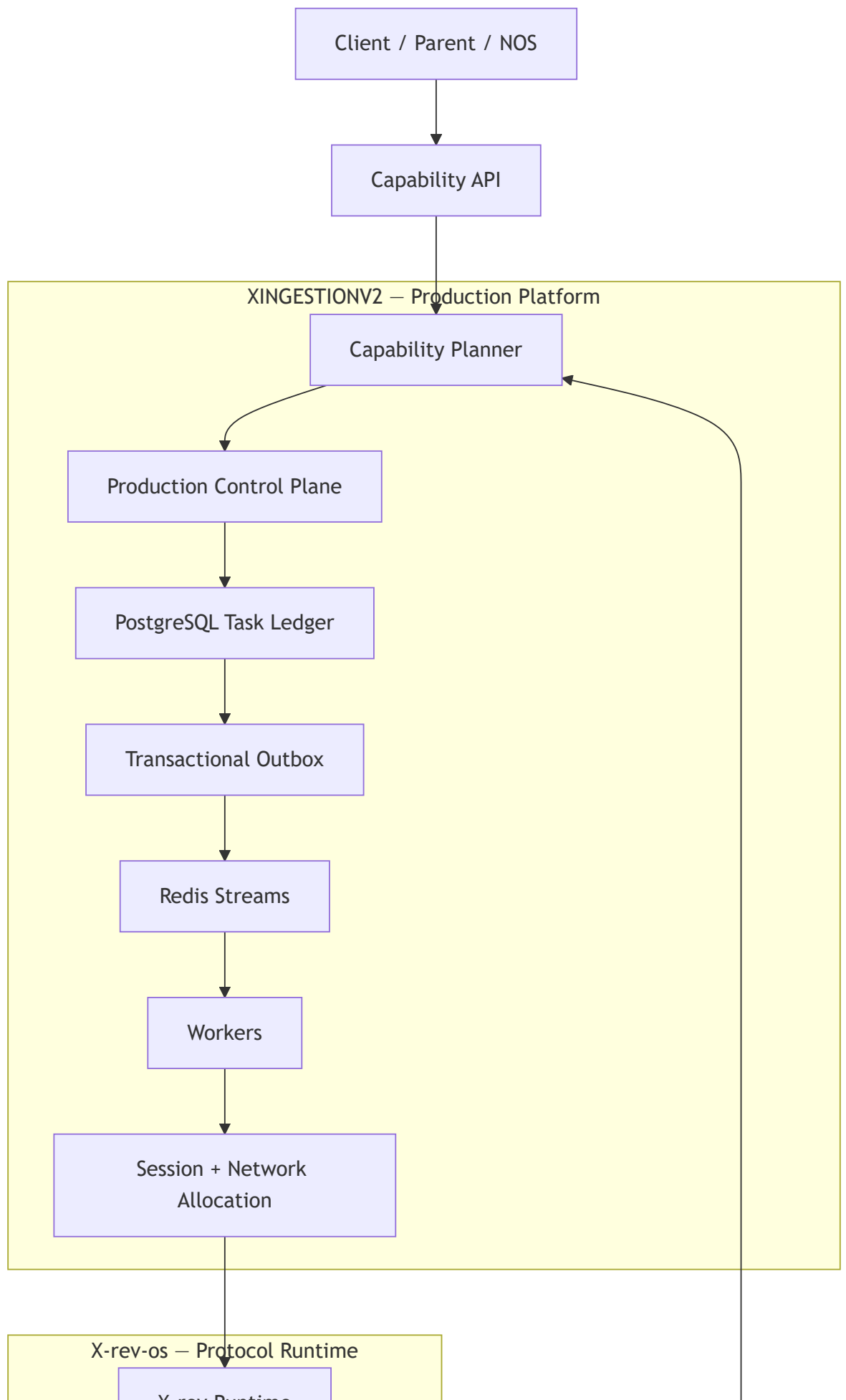


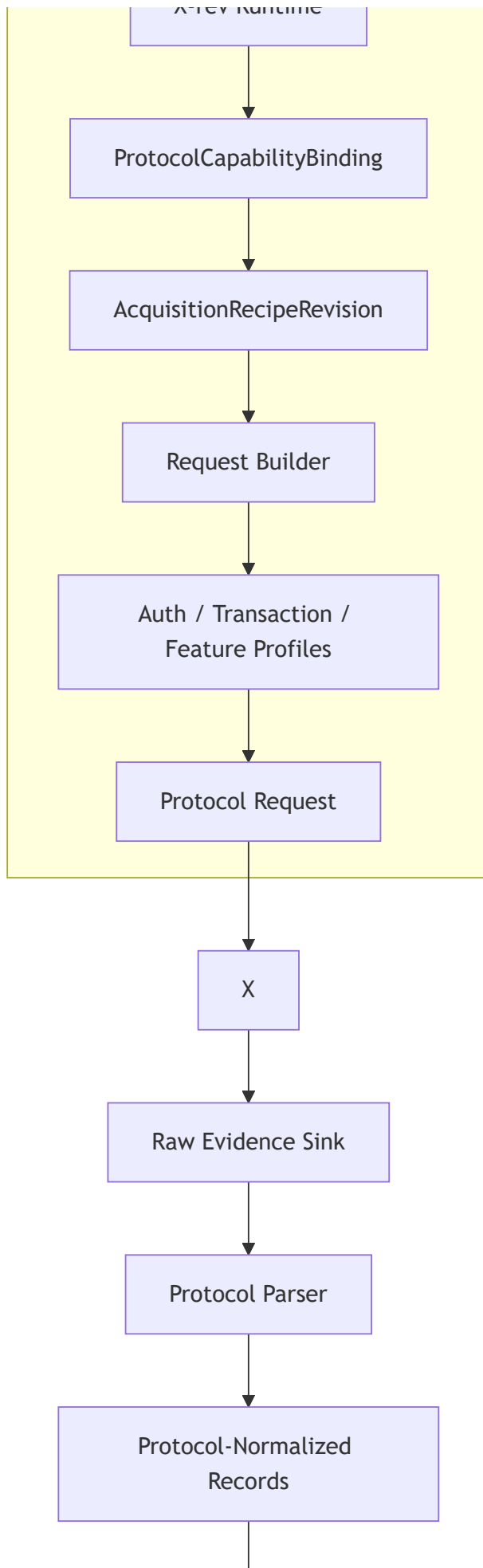
Why this is better:

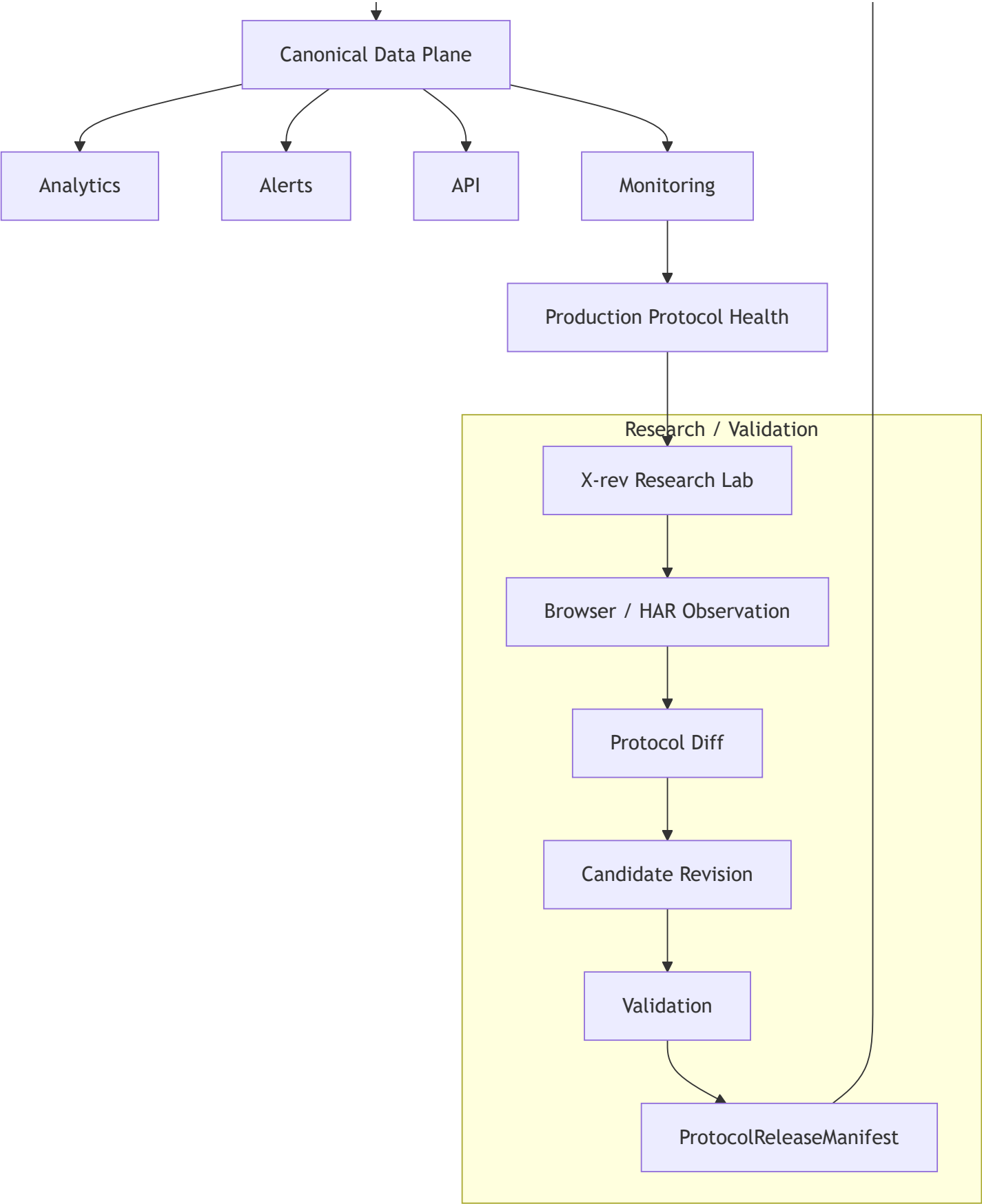
An endpoint can remain unchanged while its parser changes. A parser can remain unchanged while pagination changes. Authentication behavior can change independently from the GraphQL operation.

A single endpoint version is therefore too coarse.

4. Final Whole-System Architecture







5. Stable Product Capabilities

Production clients must request **capabilities**, not X endpoint names.

Examples:

```
SEARCH_TWEETS
TWEET_BY_ID
TWEET_REPLIES
TWEET_QUOTES
USER_LOOKUP
USER_TIMELINE
FOLLOWERS
FOLLOWING
LIST_TIMELINE
COMMUNITY_TIMELINE
MONITOR_QUERY
MONITOR_USER
```

A capability defines **what the caller needs**, not how X currently provides it.

Example:

```
Capability: SEARCH_TWEETS
Contract version: 1
```

Inputs:

- query
- product
- cursor
- page_size

Expected behavior:

- real tweet IDs
- text
- author
- timestamps
- supported engagement metadata
- opaque pagination cursor
- acquisition provenance

The capability contract must not contain:

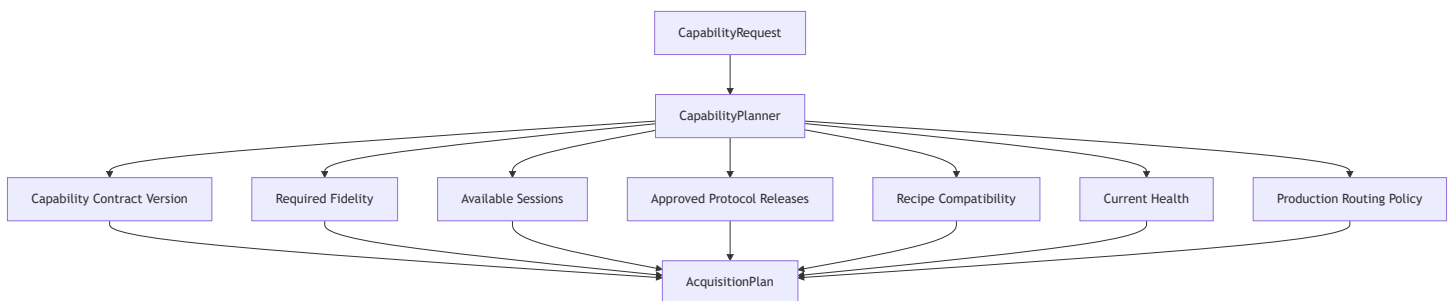
- X GraphQL operation IDs;
- X request URLs;
- feature flag bundles;
- parser implementation details;
- cookies;
- CSRF tokens;
- browser selectors;
- Twikit or twscrape types.

This allows the X protocol implementation to change without breaking the client-facing API.

6. Capability Planner

The `CapabilityPlanner` converts a stable `CapabilityRequest` into an executable production plan.

Conceptually:



An `AcquisitionPlan` should identify:

- capability ID;
- contract version;
- approved X-rev release;
- acquisition recipe revision;
- required auth class;
- cursor/page policy;
- traffic priority;
- production routing information.

The planner does **not** contain cookies, passwords, or raw X operation definitions.

7. Production Control Plane

The production control plane is owned entirely by XINGESTIONV2.

Its job is to guarantee that ingestion work is:

- durable;
- recoverable;
- auditable;
- retryable;
- scalable;
- crash-safe.

7.1 PostgreSQL is the durable authority

PostgreSQL is the source of truth for task state.

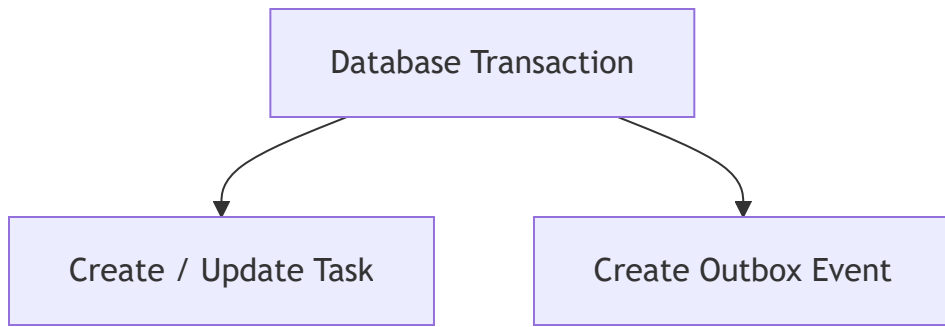
Tasks are not considered real merely because they appear in Redis.

Typical states include:

CREATED
ENQUEUED
RUNNING
RETRY_SCHEDULED
DONE
DEAD_LETTER

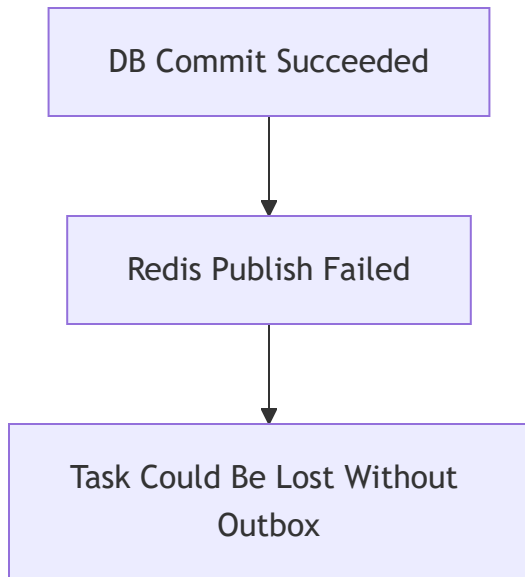
7.2 Transactional outbox

A task and its publication intent are committed atomically.



A dispatcher later publishes the outbox event to Redis Streams.

This prevents the classic failure:



7.3 Redis Streams

Redis is delivery infrastructure.

It provides:

- consumer groups;
- pending messages;
- worker distribution;
- reclaim behavior.

It is **not** the authoritative task database.

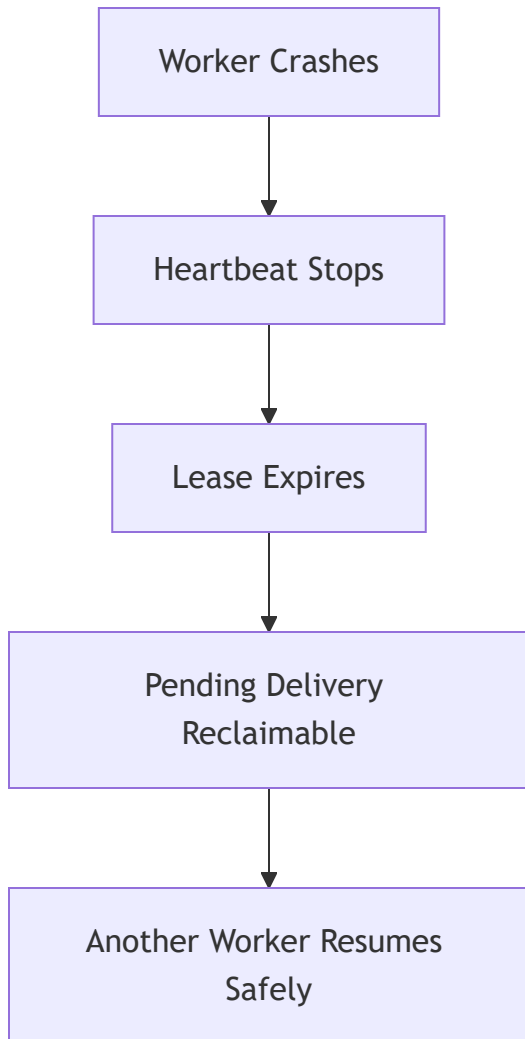
7.4 Worker leases

Workers do not simply "take a task forever".

A worker obtains a lease.

The lease must be renewed through heartbeat logic.

If a worker dies:



7.5 Fencing

A stale worker must not be able to commit after losing ownership.

All writes must be fenced by task identity, delivery generation, lease owner/token, and valid task state.

8. Retry Ownership

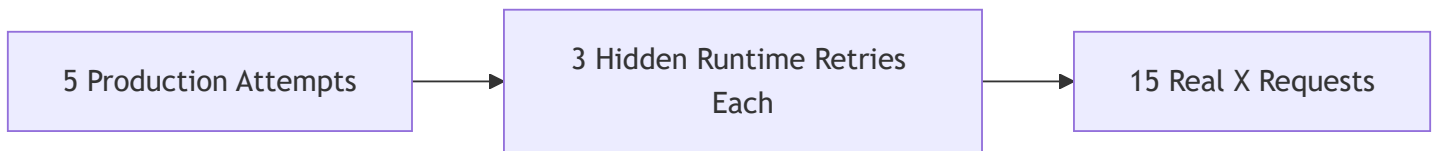
This rule is critical:

XINGESTIONV2 exclusively owns production retries.

The X-rev production runtime performs **zero automatic retries**.

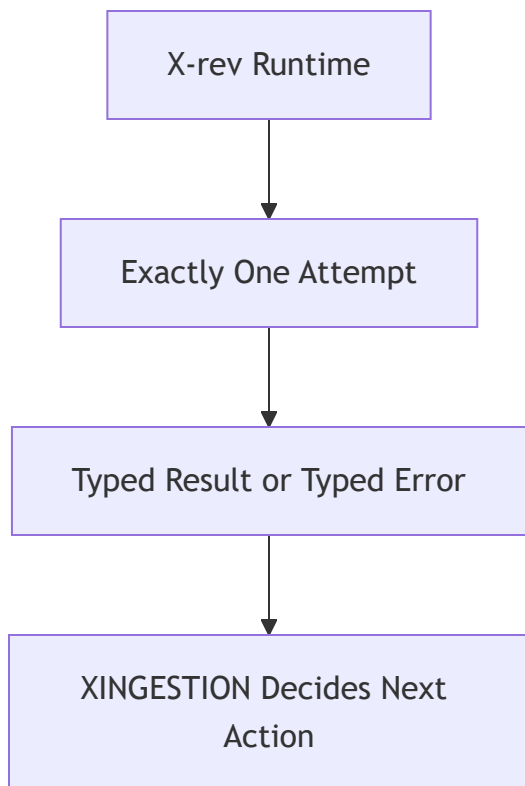
Why:

If XINGESTION allows 5 task attempts and X-rev secretly retries each request 3 times:



This destroys visibility and pacing control.

Instead:



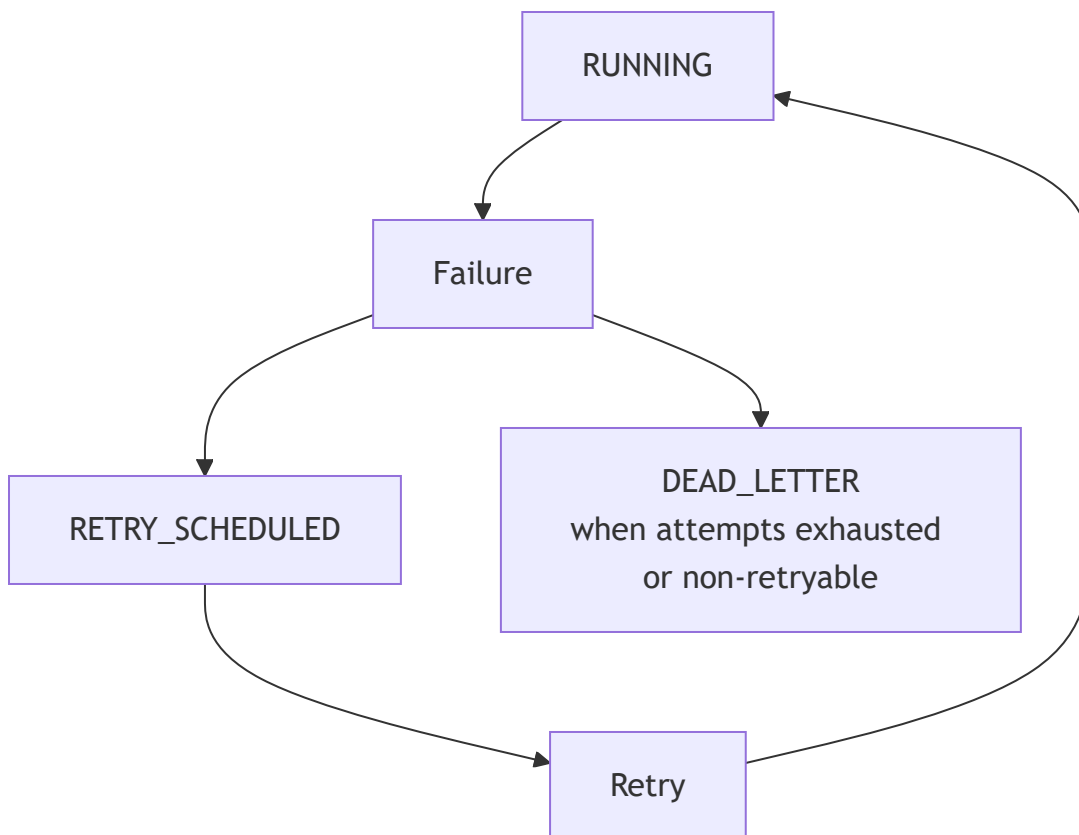
Typed error metadata can include:

```
retry_disposition = NEVER
retry_disposition = MAY_RETRY
retry_disposition = RETRY_AFTER
retry_after = ...
scope_hint = SESSION / OPERATION / SHARED / UNKNOWN
```

XINGESTION applies durable scheduling, backoff, jitter, retry generations, and attempt limits.

9. Dead-Letter and Replay

When retry attempts are exhausted:



Dead-letter records must preserve:

- original task ID;
- failure class;
- attempts;
- delivery generation;
- protocol provenance;

- timestamps;
- safe diagnostic metadata.

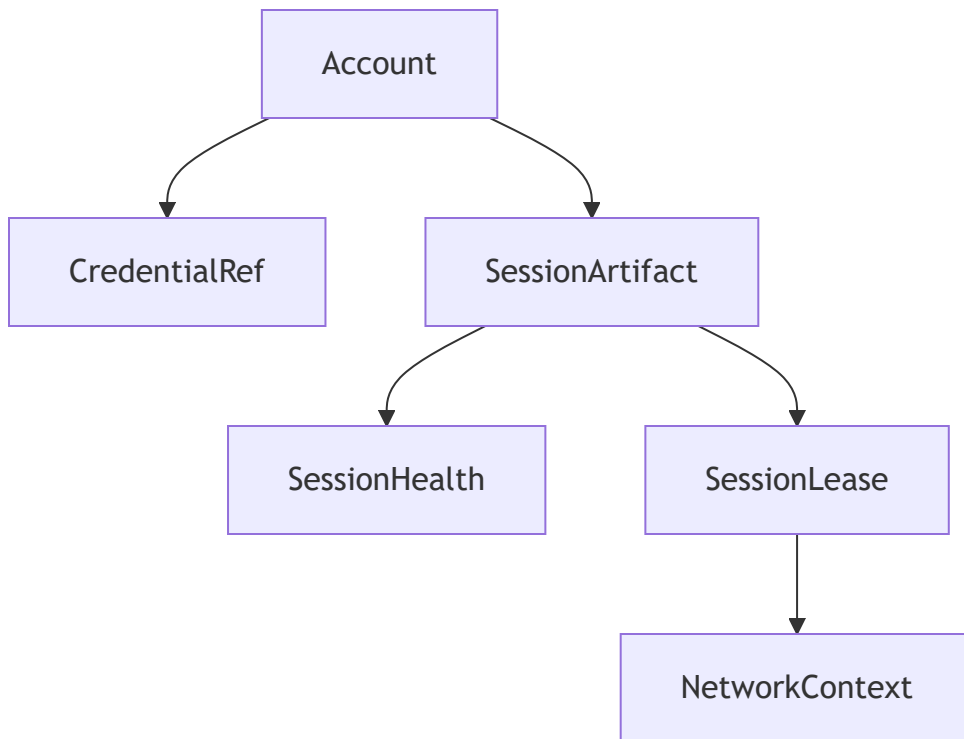
Replay must be:

- selective;
- explicit;
- auditable;
- lineage-preserving.

A replayed task is a new task linked to its dead-letter origin.

10. Account, Credential, Session, and Network Model

These concepts must remain separate.



10.1 Account

Non-secret identity metadata for an authorized account.

10.2 CredentialRef

A reference to long-lived sensitive material stored in a proper secret backend.

Never place raw secrets in task payloads.

10.3 SessionArtifact

Represents usable authenticated session material.

Possible data may include:

- cookies;
- session-specific metadata;
- optional protocol-required authorization material.

The sensitive values remain secret.

10.4 SessionHealth

Examples:

```
HEALTHY
DEGRADED
AUTH_EXPIRED
CHALLENGED
LOCKED
DISABLED
UNKNOWN
```

Health is not the same thing as lease ownership.

10.5 SessionLease

Controls temporary production use of a session.

It includes:

- owner;

- expiry;
- concurrency rules;
- fencing token;
- reclaim behavior.

10.6 NetworkContext

Represents the network route selected by production.

It may include:

- direct route;
- approved proxy/gateway;
- region;
- transport metadata.

XINGESTION selects the route.

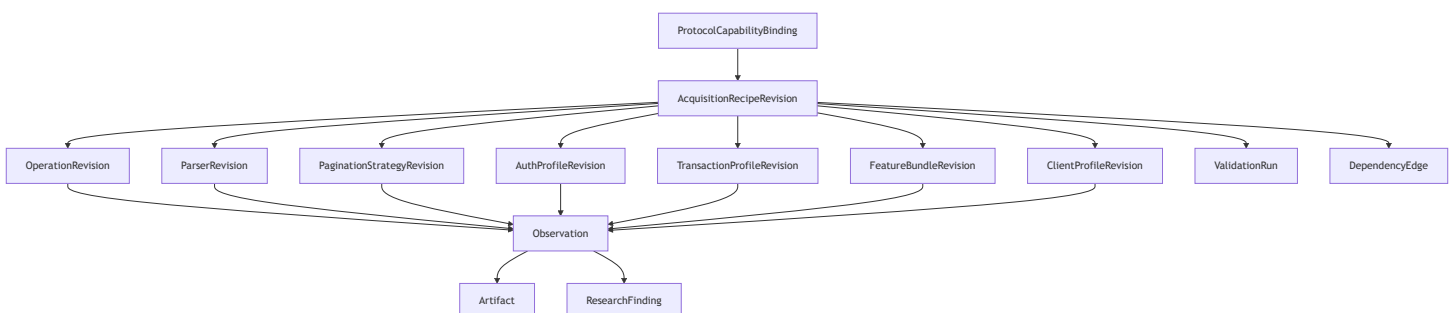
X-rev only receives the provided `NetworkContext` /transport and constructs the X-specific request correctly.

11. X-rev-os Protocol Model

X-rev-os is not an endpoint list.

It is an evidence-backed protocol dependency system.

Core concepts:



12. Immutable Revision Model

Research objects can be mutable only while they are explicitly drafts.

Examples:

```
DraftOperation
DraftRecipe
DraftParser
```

Once published:

```
OperationRevision
AcquisitionRecipeRevision
ParserRevision
```

they are immutable.

If behavior changes, create a new revision.

This provides reproducibility.

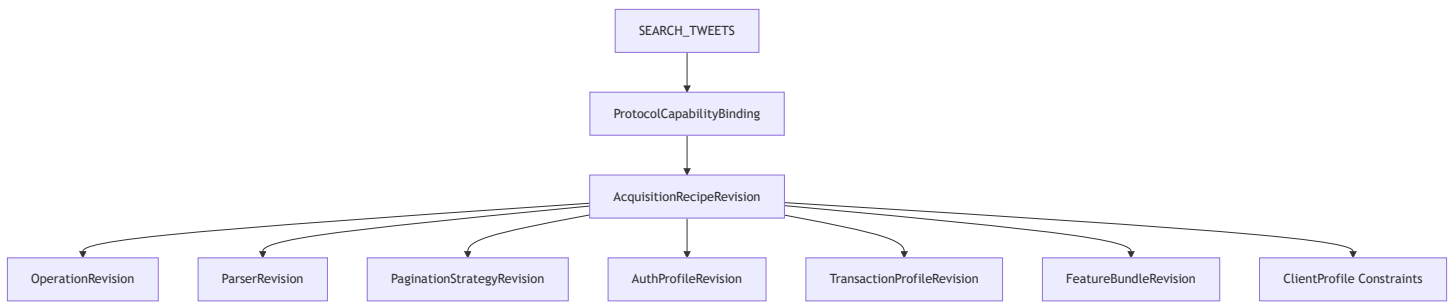
A historical production run should always be able to answer:

```
Which operation definition?
Which parser?
Which pagination strategy?
Which auth logic?
Which feature bundle?
Which runtime?
Which protocol bundle?
```

13. Acquisition Recipe

A capability is executed through an `AcquisitionRecipeRevision` .

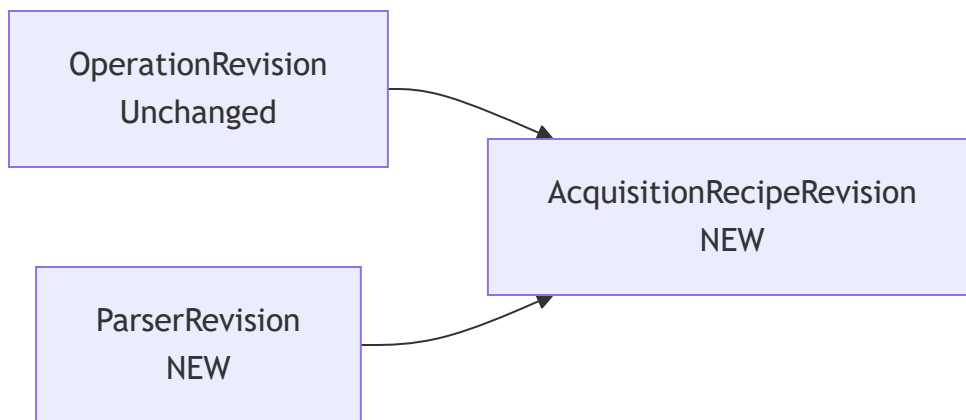
Example:



This prevents unrelated protocol components from being incorrectly versioned together.

Example:

If only the parser changes:



The previous complete-recipe validation does not automatically transfer.

14. Three Separate State Dimensions

Do not collapse all protocol state into "healthy/unhealthy".

14.1 Evidence maturity

Answers:

How strongly do we know this protocol fact?

REFERENCE
OBSERVED
INFERRED
VALIDATED

14.2 Validation freshness

Answers:

How recent is the validation for this exact composition?

NEVER_VALIDATED
CURRENT
AGING
EXPIRED

14.3 Operational health

Answers:

What are recent observations telling us operationally?

UNKNOWN
ACTIVE
DEGRADED
STALE
QUARANTINED
RETIRED

A recipe can legitimately be:



Meaning:

It worked and was previously proven, recent validation is old, but production has not observed a current failure.

15. Protocol Observation

X-rev researches X through authorized observable behavior.

Observation sources may include:

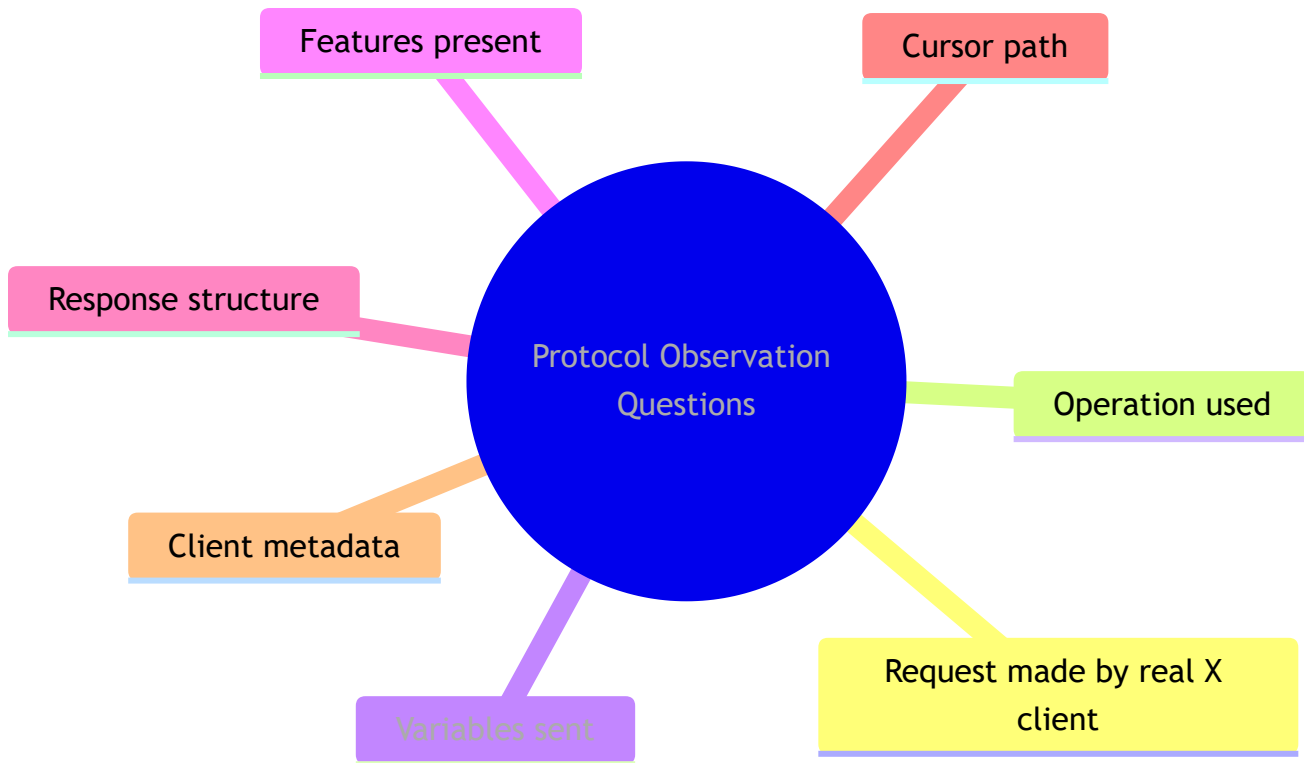
- browser network traffic;
- HAR exports;
- CDP/browser instrumentation;
- sanitized historical fixtures;
- direct-client captures;
- public client artifact analysis;
- comparison with reference libraries.

The browser is a research tool, not the core production transport.

That is a key architectural decision.

15.1 Browser role

Browser/HAR observation helps answer:



15.2 Production role

Production should eventually use the validated first-party X-rev direct runtime rather than launching a research browser for each collection task.

16. Raw Evidence

Raw evidence is a first-class durability requirement.

16.1 Research raw evidence

Owned by X-rev-os.

It remains:

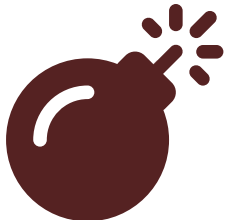
- local;
- gitignored;
- protected from accidental commit.

16.2 Production raw evidence

Owned by XINGESTIONV2.

The X-rev runtime receives an injected `RawEvidenceSink` .

Flow:



Syntax error in text
mermaid version 11.15.0

If production raw persistence is required and fails:

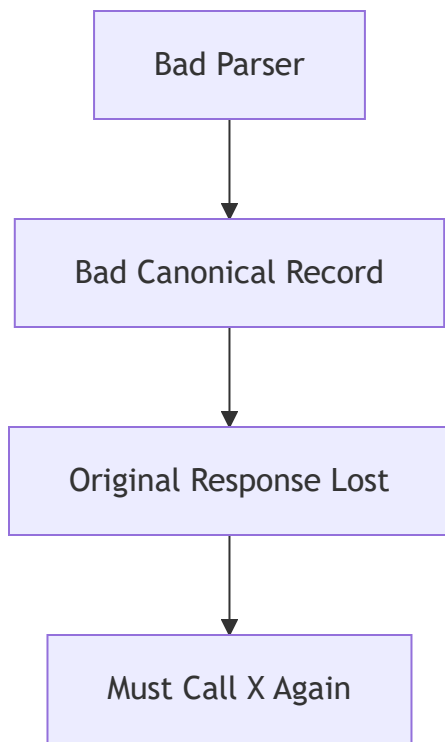
```
RAW_EVIDENCE_PERSISTENCE_FAILED
```

The system must not silently declare successful safe acquisition.

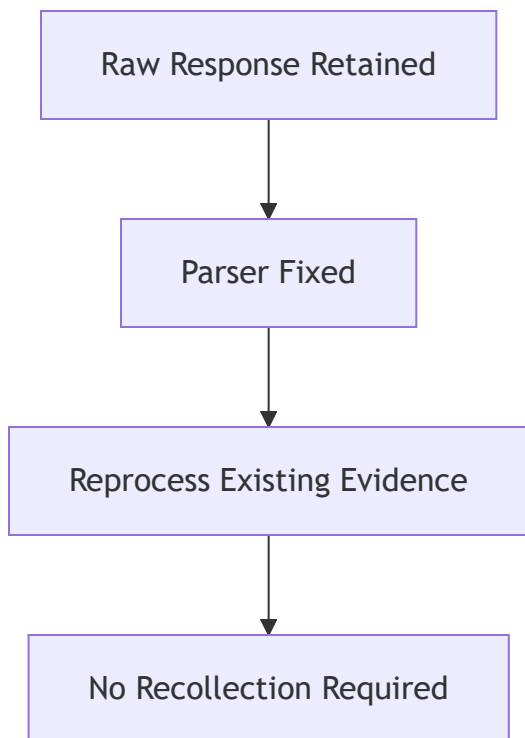
17. Why Raw Evidence Must Be Stored Before Normalization

Suppose the parser has a bug.

Without raw evidence:



With raw evidence:



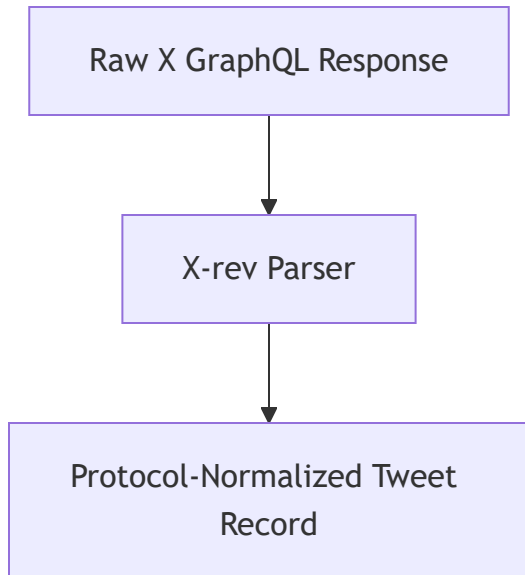
This reduces:

- unnecessary upstream requests;
- cost;
- data loss;
- debugging uncertainty.

18. Parser and Protocol-Normalized Output

X-rev parses protocol-specific response structures into protocol-normalized capability records.

Example:



Possible fields:

```
tweet_id
author_id
username
text
created_at
reply_count
repost_count
like_count
quote_count
media
conversation_id
```

XINGESTION then converts these into the larger canonical production data model.

19. Canonical Data Plane

XINGESTION owns:

- canonical entities
- relationship edges
- time-varying observations
- normalization versions
- reprocessing
- provenance

Do not treat everything as a tweet document.

Examples:

- Tweet
- User
- List
- Community
- RelationshipEdge
- EngagementObservation
- ProfileObservation

19.1 Object identity

Platform object IDs are identity.

Text hashes must not be used to destructively merge distinct objects.

19.2 Engagement counters

Engagement counts are observations over time.

Do not model:

likes +100

as if the platform emitted a guaranteed event.

Instead:

```
tweet_id = 123  
captured_at = T1  
likes = 100
```

```
tweet_id = 123  
captured_at = T2  
likes = 160
```

19.3 Time semantics

Keep separate:

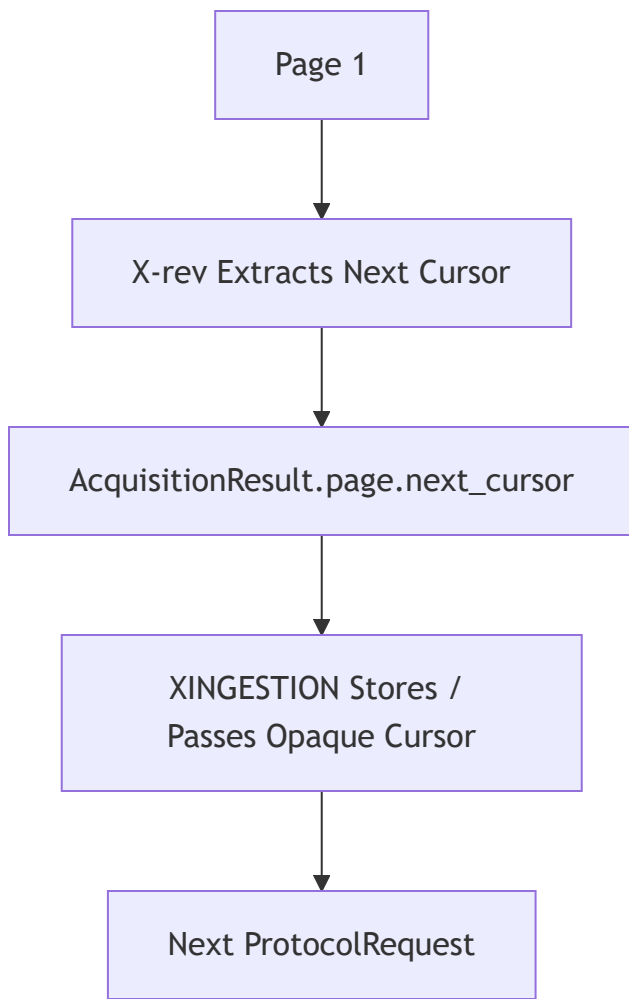
```
source_created_at  
captured_at  
first_seen_at  
last_seen_at  
source_updated_at  
normalized_at
```

20. Pagination

Pagination belongs to X-rev because cursor meaning is protocol-specific.

XINGESTION treats cursors as opaque values.

Example:



X-rev validates:

- cursor extraction;
- cursor loops;
- duplicate pages;
- missing cursor;
- empty continuation;
- page semantics.

21. Typed Protocol Errors

The runtime should provide structured errors such as:

SESSION_INVALID
SESSION_CHALLENGED
AUTH_ATTACHMENT_INVALID
TRANSPORT_TIMEOUT
NETWORK_FAILURE
HTTP_SERVER_FAILURE
RATE_LIMITED
TEMPORARY_UNAVAILABLE
OPERATION_NOT_FOUND
OPERATION_CONTRACT_CHANGED
FEATURE_OR_CONFIG_CHANGED
SHARED_TRANSACTION_PROFILE_CHANGED
CLIENT_BUILD_CHANGED
RESPONSE_SCHEMA_VARIANT
PARSER_FAILURE
PAGINATION_CURSOR_MISSING
PAGINATION_CURSOR_LOOP
PAGINATION_EMPTY_CONTINUATION
RAW_EVIDENCE_PERSISTENCE_FAILED
ACCESS_NOT_AUTHORIZED
OBJECT_NOT_FOUND
UNSUPPORTED_CAPABILITY
BUNDLE_RUNTIME_INCOMPATIBLE
UNKNOWN_PROTOCOL_FAILURE

This enables intelligent production behavior without putting protocol-specific implementation inside the worker.

22. Failure Isolation

The system is not designed around the fantasy that nothing breaks.

X changes.

The target is:

A failure should affect only its actual dependency graph.

Examples:

One operation changes

SEARCH_TWEETS degraded
USER_TIMELINE unaffected
task database unaffected
analytics unaffected

Parser breaks

raw evidence retained
normalization fails
no need to recollect

One session expires

session quarantined
other valid sessions continue

Browser research tooling breaks

research slows
validated production release keeps running

Analytics fails

acquisition continues
analytics repaired later

Redis fails

PostgreSQL remains durable authority
delivery can be reconstructed/redriven

23. Protocol Validation

A protocol capability is not validated merely because one HTTP request returned 200.

Validation belongs to the exact complete `AcquisitionRecipeRevision` .

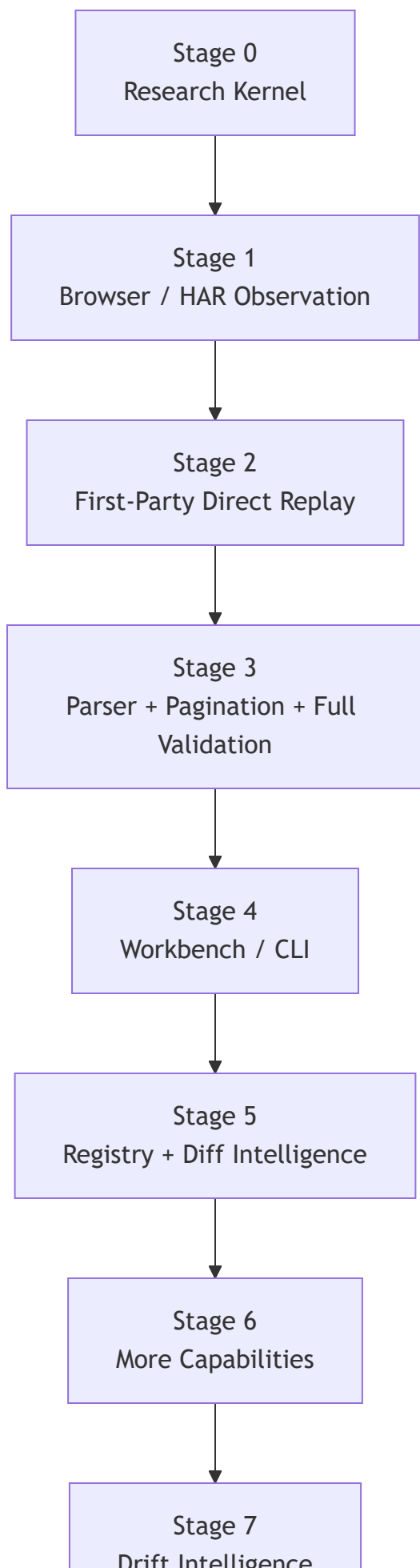
A validation record should include:

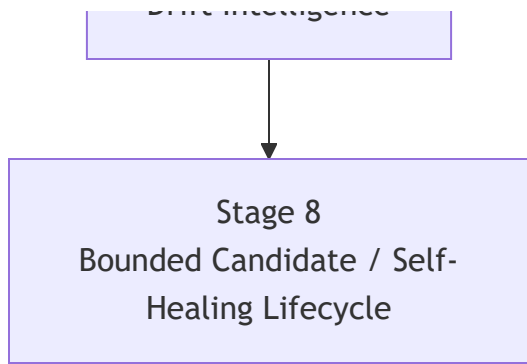
```
capability_id
recipe_revision
composition_hash
runtime_version
protocol_bundle_version
observation context
validation input/query
raw evidence references
result metrics
pass/fail
reasons
```

If a dependency changes, create a new composition and validate again.

24. Validation Lifecycle

For an initial search capability:





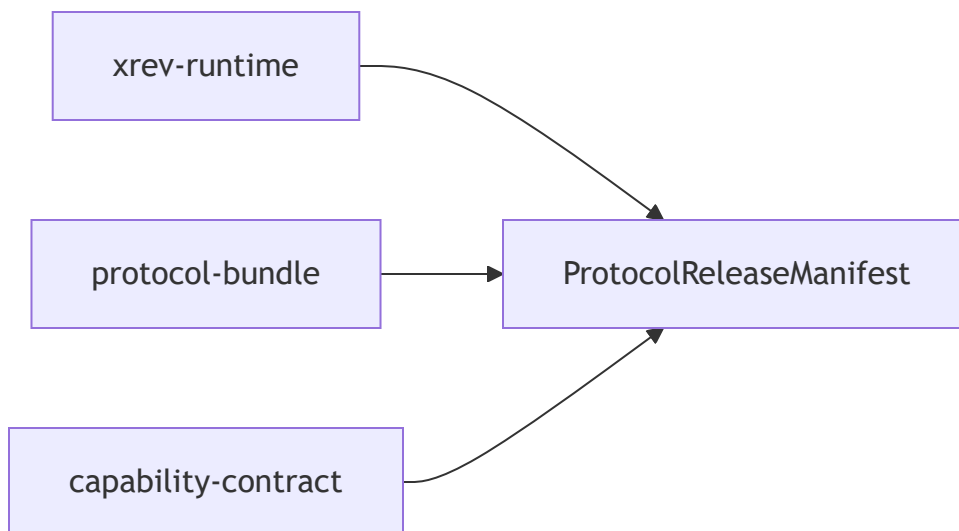
The key principle:

Do not expand breadth before one complete vertical slice proves the architecture.

25. Protocol Release Model

Production must not consume floating research state.

Version independently:



Then bind them through:

`ProtocolReleaseManifest`

A release manifest should contain:

runtime_version
protocol_bundle_version
capability_contract_version
source_git_commit
checksums
validated recipes
compatibility constraints

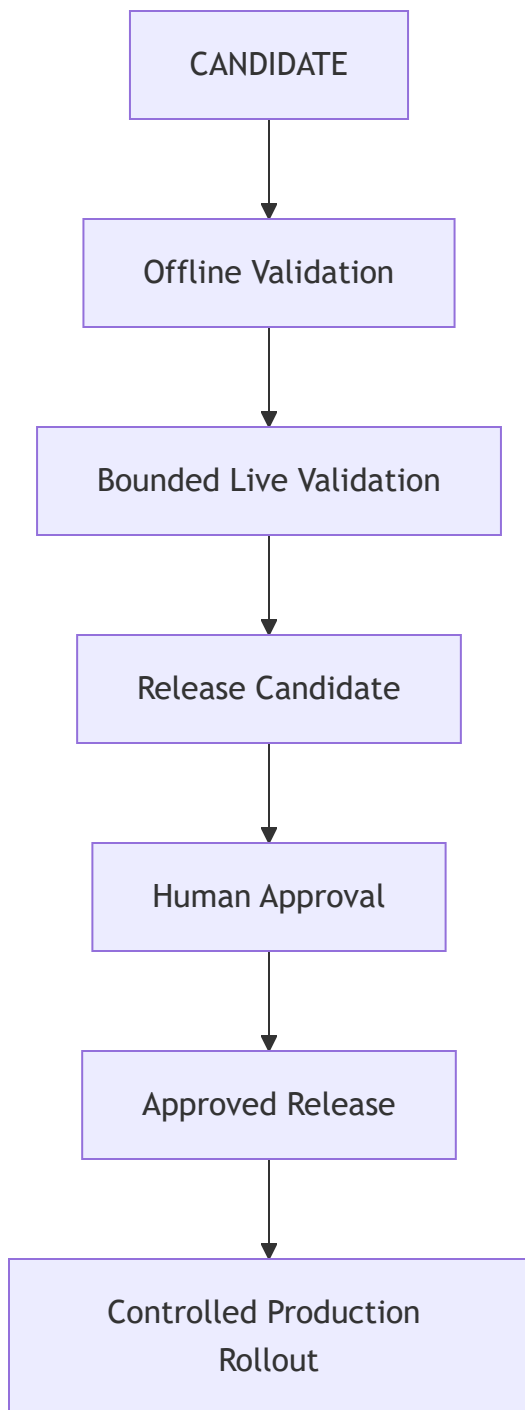
Production pins an exact approved release.

Never:

use latest X-rev automatically

26. Release Promotion Lifecycle

Initial safe lifecycle:



Later, the platform may automate failover to an **already approved compatible alternate**.

It should not autonomously invent arbitrary protocol definitions inside production workers.

27. Drift Detection

When a previously approved protocol route starts failing, production should produce an investigation package.

Example contents:



X-rev uses this to investigate:

- What changed?
- Operation?
- Feature bundle?
- Client transaction metadata?
- Auth profile?
- Parser?
- Pagination?
- Client build?
- Only one session?
- Global behavior?

With only one session/context, do not overclaim global scope.

Use explicit unknown classification.

28. Monitoring and Incremental Ingestion

Monitoring should not mean spawning thousands of independent scrape loops.

Use persistent subscriptions.

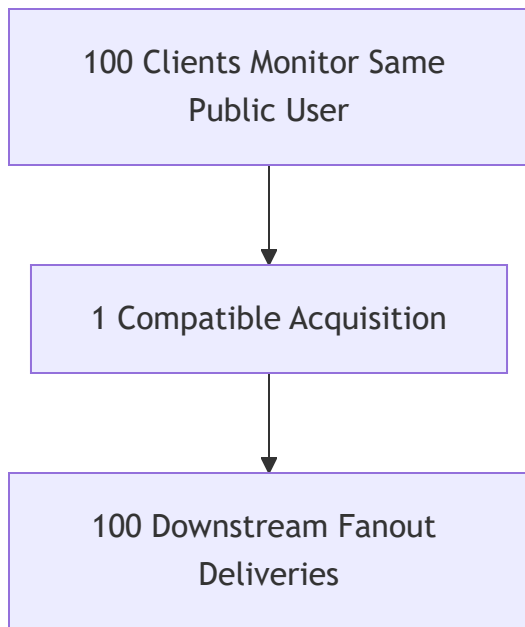
Example model:

```
subscription_id
canonical target/query
cadence
freshness objective
priority
watermark/cursor
next_due_at
catch-up policy
consumer targets
```

The scheduler should support:

- acquisition coalescing;
- deduplication;
- watermarks;
- incremental continuation;
- gap detection;
- bounded backfills;
- outage catch-up;
- jitter;
- priority;
- backpressure.

Example:



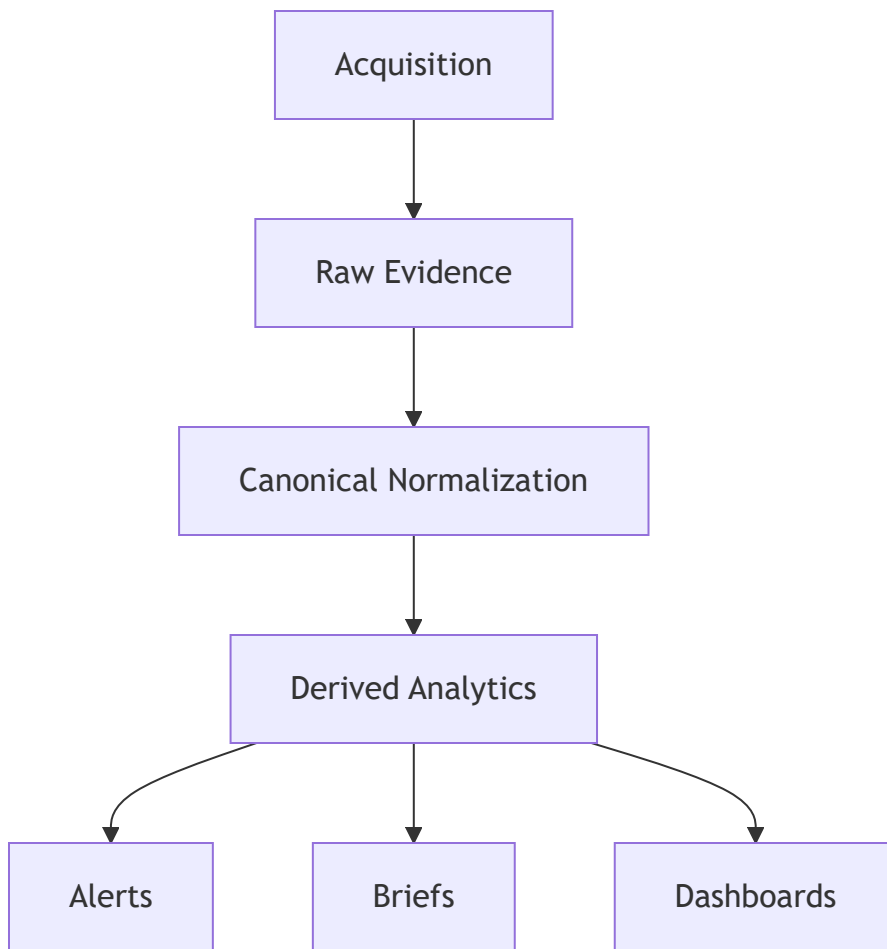
Where visibility/auth semantics differ, requests must not be incorrectly coalesced.

29. Analytics, Alerts, and Briefs

These are downstream consumers.

They must not be part of the critical acquisition transaction.

Correct architecture:



If an LLM brief service fails:

ingestion continues

No upstream recollection should be required.

30. Northbound API

External clients should interact with stable product abstractions.

They should not know:

- Redis stream names;
- task database tables;
- GraphQL operation IDs;
- X-rev research database paths;
- cookies;

- parser internals;
- feature bundles.

Possible API concepts:

```
POST /capabilities/search-tweets
POST /jobs
GET /jobs/{id}
GET /results/{id}
POST /monitors
GET /monitors/{id}
```

Final API transport/auth depends on parent-system requirements.

31. Security Model

Authentication material is sensitive.

Never commit:

```
passwords
cookies
auth tokens
CSRF token values
TOTP secrets
email credentials
proxy credentials
```

Production secrets should live behind a:

```
SecretStore / KMS / Vault / approved secret backend
```

Durable tasks store only references or safe pseudonymous metadata.

The runtime receives ephemeral secret material only when required to execute an authorized request.

Secret-bearing fields must not appear in:

- logs;
- repr/debug output;
- committed fixtures;
- analytics;
- investigation packages;
- safe provenance.

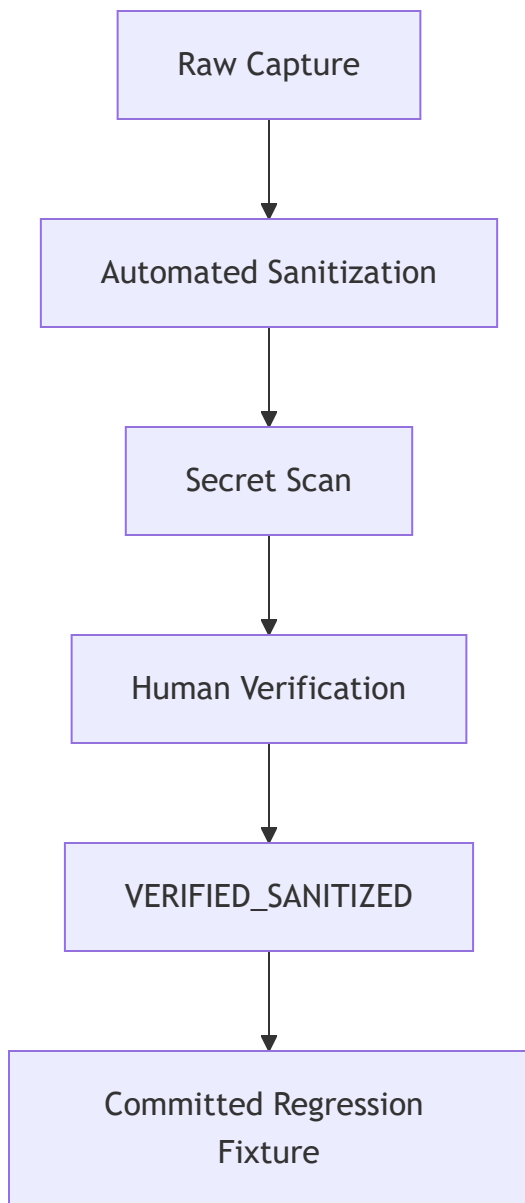
32. Sanitized Fixtures

Committed protocol fixtures must be sanitized.

Requirements:

- deterministic transformation;
- stable referential pseudonymization;
- removal of secret material;
- transformation provenance;
- hashes;
- transform version;
- explicit verification state.

Example:



Automated tooling must not self-certify sensitive real captures as safe without the required review policy.

33. Observability

Production should expose metrics for:

queue depth
oldest task age
outbox lag
tasks completed
tasks retried
tasks dead-lettered
replays
worker leases
session availability
session concurrency
session cooldowns
capability success rate
protocol error distribution
release/recipe health
parser warnings
pagination warnings
schema fingerprints
latency
rate limiting
raw processing lag
normalization lag
monitor lag
release rollout state
API/downstream health

Logs must be:

- structured;
- correlated;
- secret-free.

34. What Is Already Implemented

Based on the current active project documentation, substantial foundations already exist.

XINGESTIONV2

Verified foundations include:

Control plane

- PostgreSQL task ledger;
- idempotency;
- delivery generations;
- transactional outbox;
- Redis Streams;
- consumer groups;
- dispatcher claiming;
- durable task state.

Worker recovery

- execution leases;
- heartbeats;
- lease renewal;
- pending message refresh/reclaim;
- stale-owner fencing;
- deterministic crash recovery.

Retry lifecycle

- durable retry scheduling;
- attempt increments;
- delivery-generation rollover;
- stale-delivery rejection;
- dead-letter archive;
- selective replay;
- replay lineage/audit.

Capability foundation

- canonical capability contracts;
- machine-readable capability artifact;
- typed capability requests;
- planner boundary;
- compatibility mapping from legacy search tasks.

X-rev-os

Implemented research/runtime foundation includes:

Research kernel

- package/CLI;
- revision models;
- evidence/artifact models;
- deterministic identity;
- local artifact storage;
- SQLite research ledger;
- sanitization;
- fixture transformation;
- secret checks;
- offline CI.

Observation

- pluggable browser observation boundary;
- Playwright-based observer;
- HAR-based observation/import path;
- raw-first storage;
- request/response correlation;
- structural fingerprints;
- operation candidate extraction;
- sanitized fixture candidate generation.

Direct replay foundation

- typed runtime concepts;
- request/session/network boundaries;
- raw evidence sink;
- one-attempt transport;
- typed protocol errors;
- zero automatic retries.

The remaining roadmap continues from this verified base.

35. What Is Still Missing

Major remaining product work includes:

X-rev-os

- complete parser/pagination/full recipe validation for the target capability;
- broader Workbench/CLI;
- historical protocol registry/diff intelligence;
- additional capabilities;
- mature drift intelligence;
- bounded candidate lifecycle.

XINGESTIONV2

- production account/session/secret/network subsystem;
- production RawEvidenceSink;
- runtime adapter consuming approved X-rev releases;
- first-party production SEARCH_TWEETS vertical slice;
- canonical normalization/reprocessing pipeline;
- production protocol telemetry;
- controlled release rollout/rollback;
- expanded capability families;
- monitoring subscriptions;
- stable northbound API;
- downstream analytics decoupling;
- deployment/security/retention hardening;
- large-scale chaos/soak/capacity testing.

36. Final Implementation Order

Recommended sequence:

1. Complete X-rev SEARCH_TWEETS validation
2. Build XINGESTION session/secret/network plane
3. Build production RawEvidenceSink
4. Build XINGESTION ↔ X-rev runtime adapter
5. Pin an approved ProtocolReleaseManifest
6. Run production SEARCH_TWEETS vertical slice
7. Decouple canonical normalization
8. Add protocol telemetry
9. Add rollout/rollback and known-approved failover
10. Expand tweet capabilities
11. Expand user/timeline capabilities
12. Expand graph/list/community capabilities
13. Add monitoring subscriptions
14. Add stable northbound API
15. Fully decouple analytics/alerts/briefs
16. Production deployment/security/retention
17. Scale/chaos/soak certification

37. Definition of Done for the Final Product

The system should not be called production-ready merely because it can fetch X data.

A serious production-ready claim requires:

Protocol

- validated exact recipe composition;
- pinned runtime/bundle/release;
- deterministic parser tests;
- pagination tests;
- drift evidence/provenance.

Control plane

- durable tasks;
- crash recovery;
- retry/dead-letter/replay;
- fencing;
- redrive strategy;
- failure injection.

Sessions

- safe secret storage;
- lease/reclaim;
- concurrency limits;
- health state;
- revocation handling.

Data

- raw evidence durability;
- canonical normalization;
- reprocessing;
- correct identity/time semantics.

Observability

- operational metrics;
- protocol metrics;
- release provenance;
- alerting.

Security

- no leaked secrets;
- least privilege;
- secret backend;

- safe logs;
- sanitized fixtures.

Operations

- migrations;
- deployment;
- retention;
- rollback;
- runbooks.

Scale

- measured capacity;
- burst behavior;
- backpressure;
- chaos tests;
- soak tests;
- documented SLOs.

38. Documentation Authority

Recommended authority model:

Product-level

FINAL_PRODUCT_SPEC.md

Answers:

What is the complete product?

X-rev-os

AGENTS.md
architecture.md
plan.md
implemented.md

Answers:

| How does protocol research and runtime work?

XINGESTIONV2

AGENTS.md
architecture.md
protocol-integration.md
plan.md
implemented.md

Answers:

| How does the production ingestion platform work?

Historical / archived

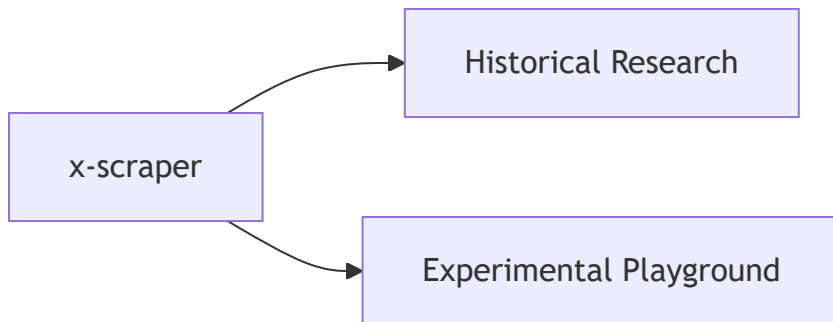
x-scraper/Twitter Research.md
XINGESTIONV2/ARCHITECTURAL_BLUEPRINT.md

These should be treated as research/history unless explicitly migrated into the active architecture.

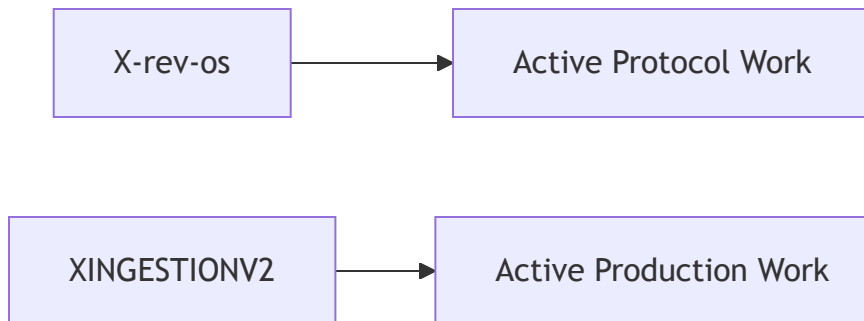
39. Recommended Cleanup

x-scraper

Keep as:



Its README should point users to:



XINGESTIONV2/ARCHITECTURAL_BLUEPRINT.md

Move or clearly mark as:

SUPERSEDED / HISTORICAL

It should not compete with:

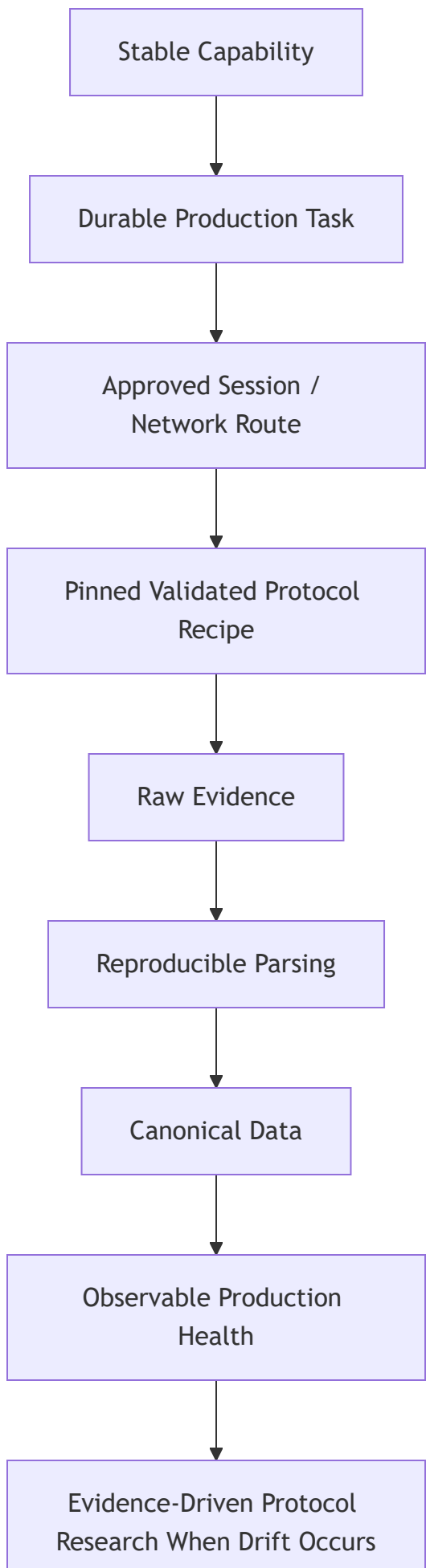
architecture.md
protocol-integration.md

40. Final Product Philosophy

The product should not be built around:

"find a working query ID and keep scraping"

It should be built around:



The core engineering principle is:

Treat X protocol behavior as a versioned, observable, replaceable subsystem—not as hard-coded scraper logic spread throughout the production application.

And the core operational principle is:

When something breaks, know exactly which layer broke, preserve enough evidence to diagnose it, and keep unrelated layers running.

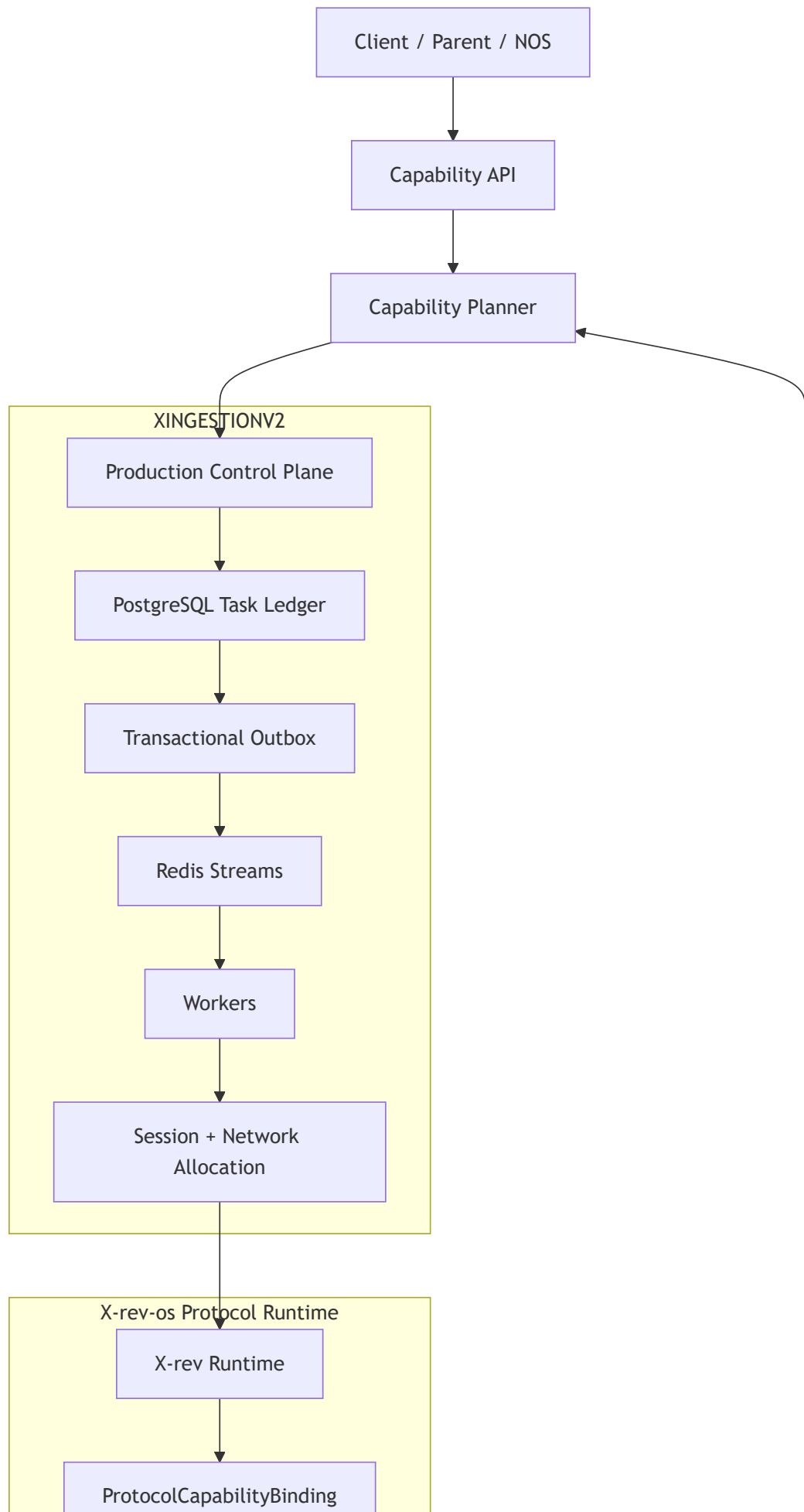
41. One-Sentence Product Definition

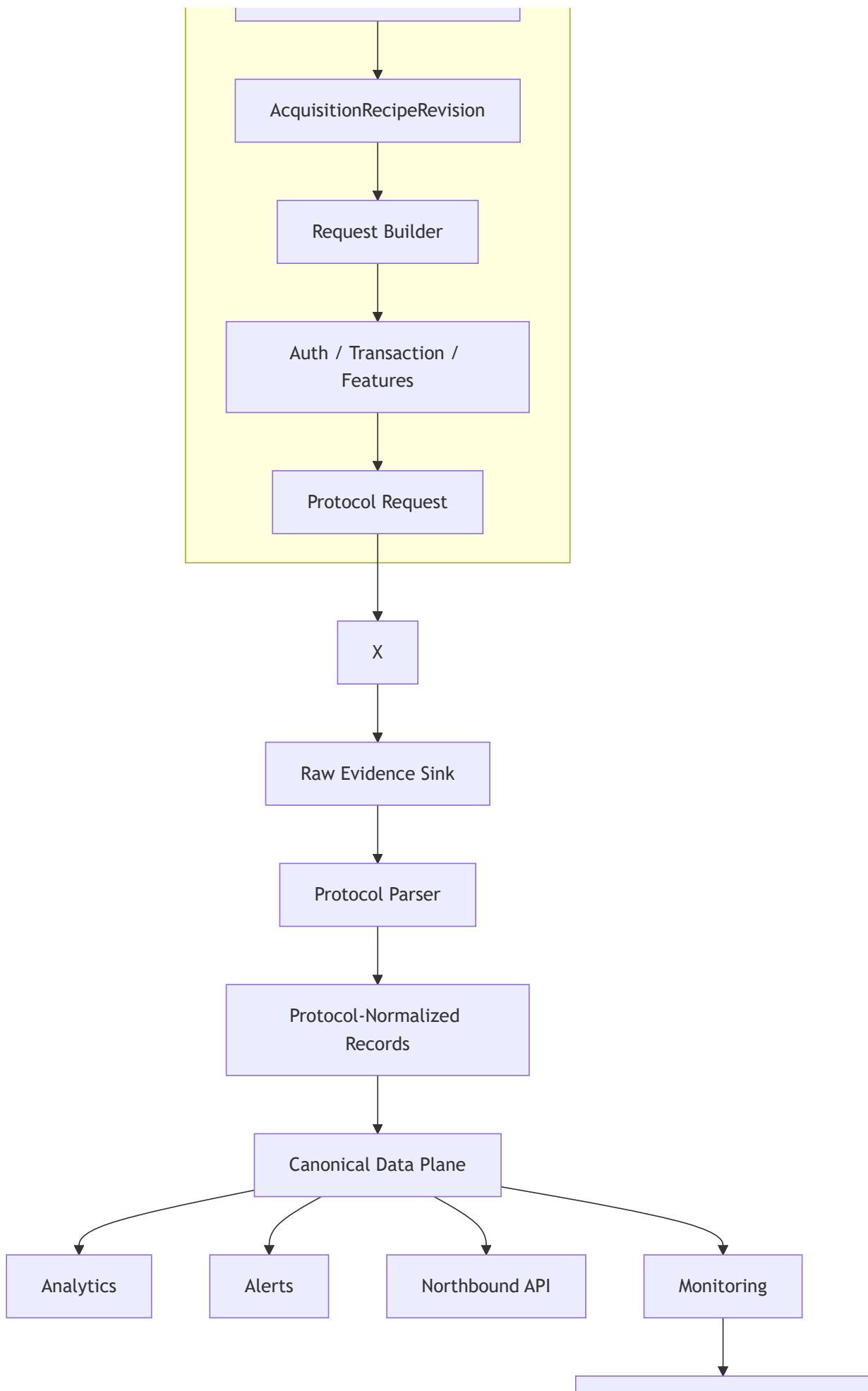
A production-grade, capability-driven X/Twitter ingestion platform that separates durable orchestration from evidence-backed protocol research, preserves raw provenance, safely manages authorized sessions, validates versioned protocol recipes, and remains diagnosable and recoverable as the upstream web protocol evolves.

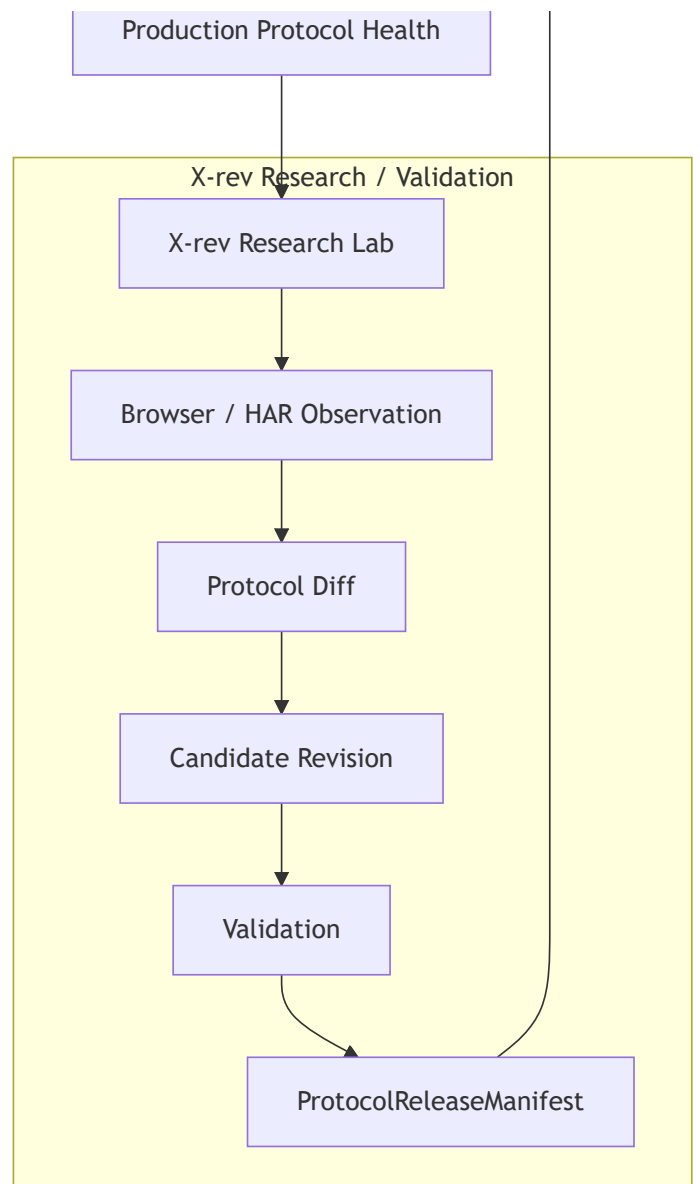
42. Mermaid Architecture Diagrams

The following Mermaid diagrams are the canonical visual companion to the architecture described above.

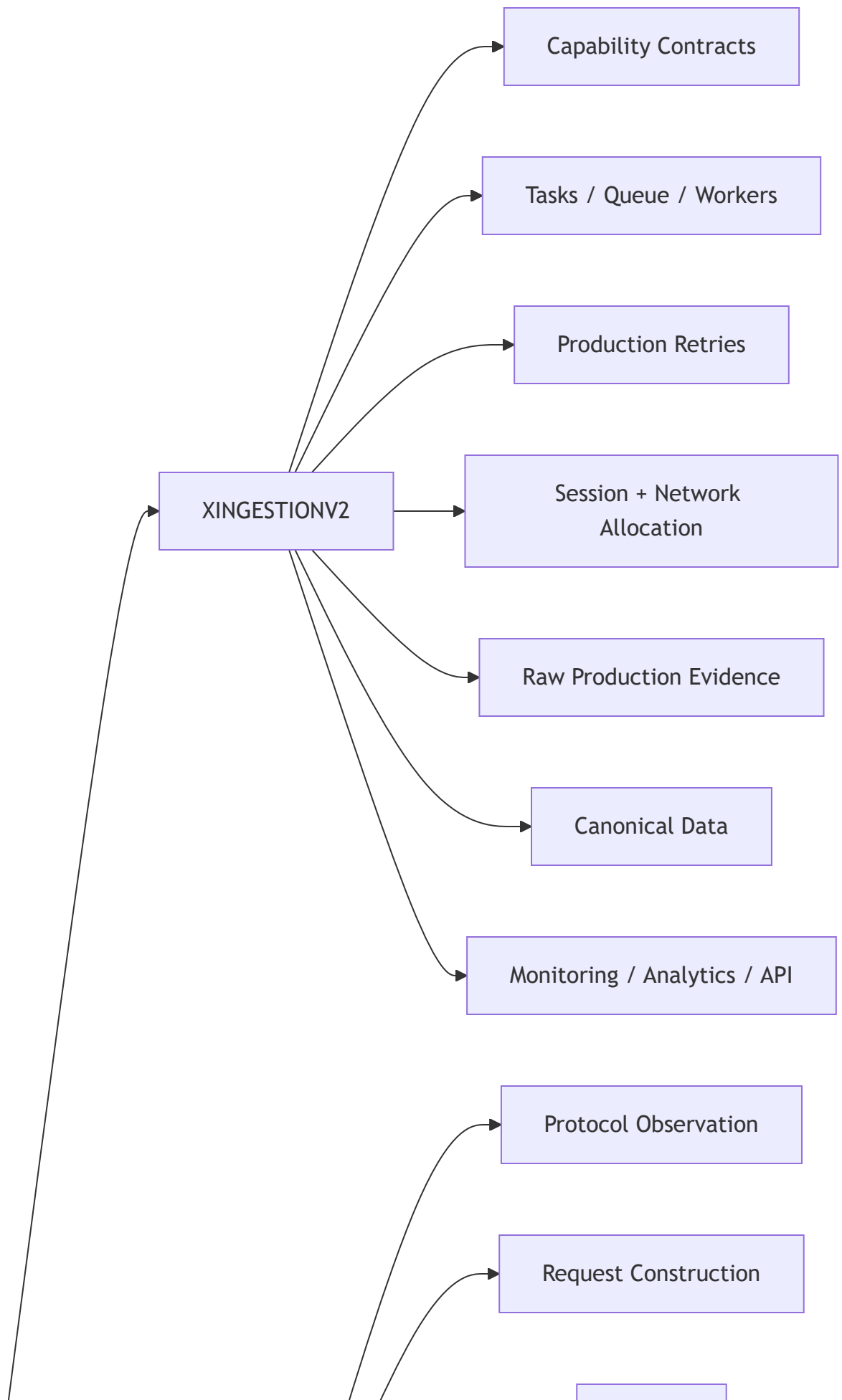
42.1 Complete Final Product Architecture

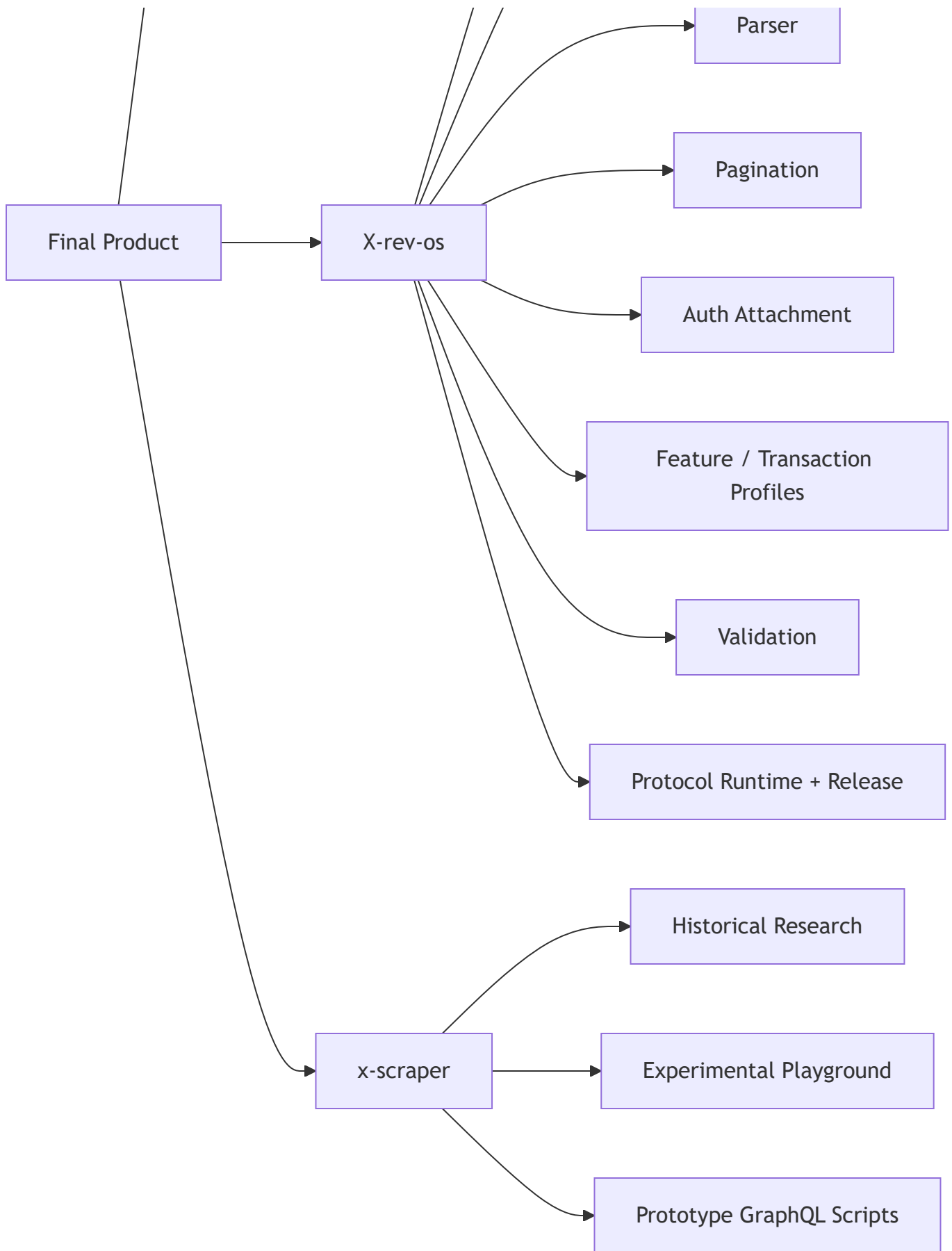




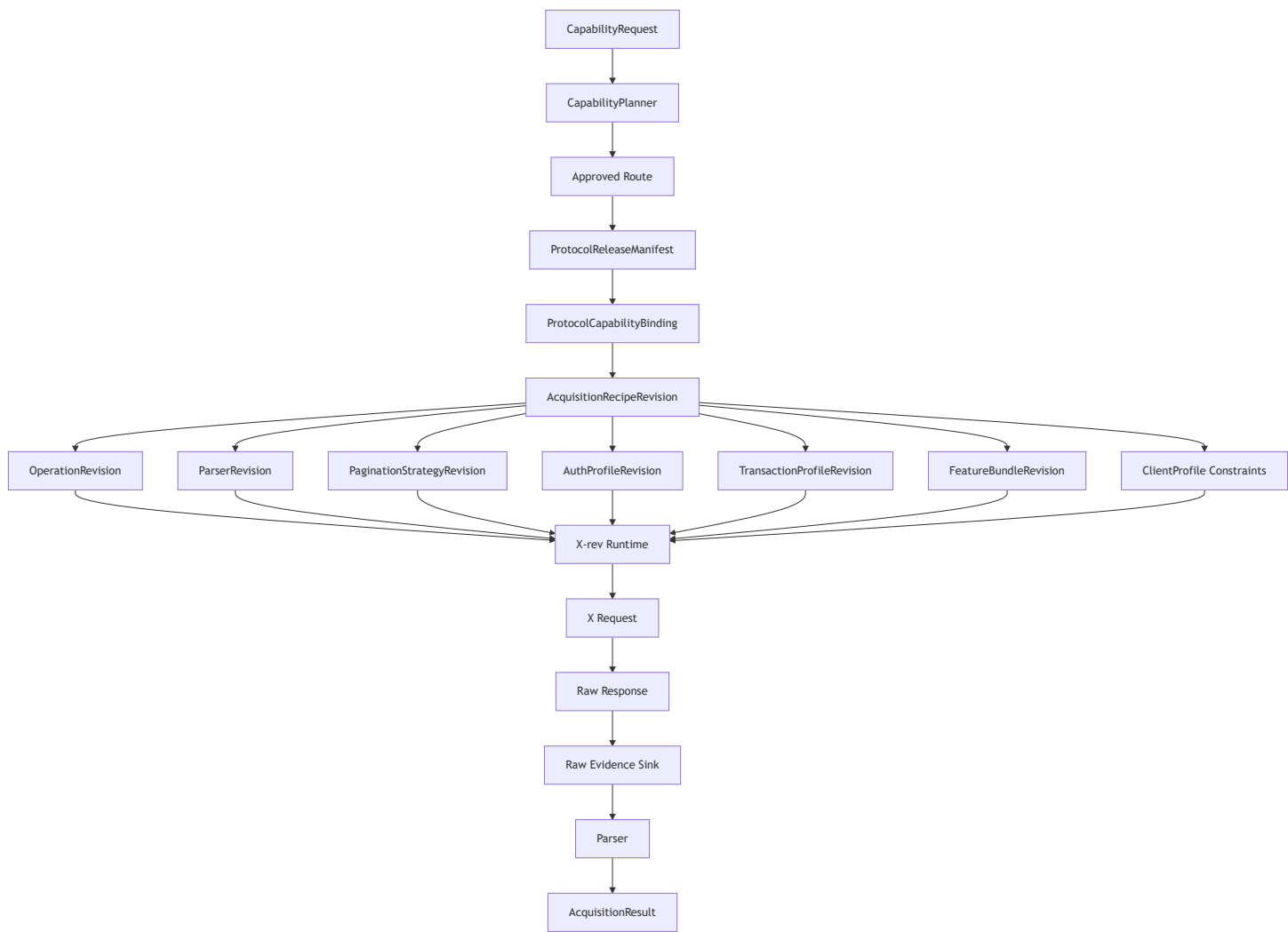


42.2 Repository Ownership

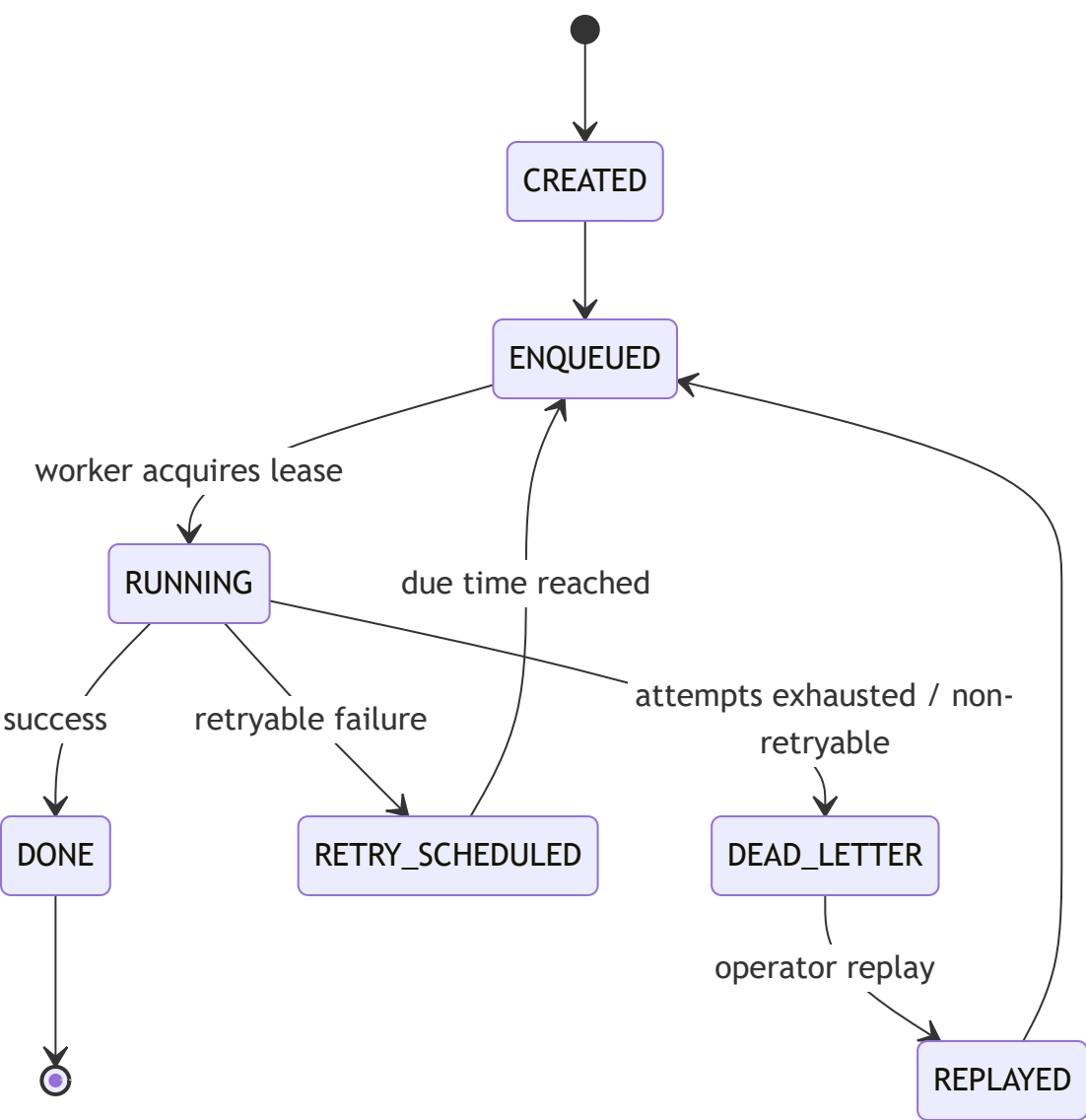




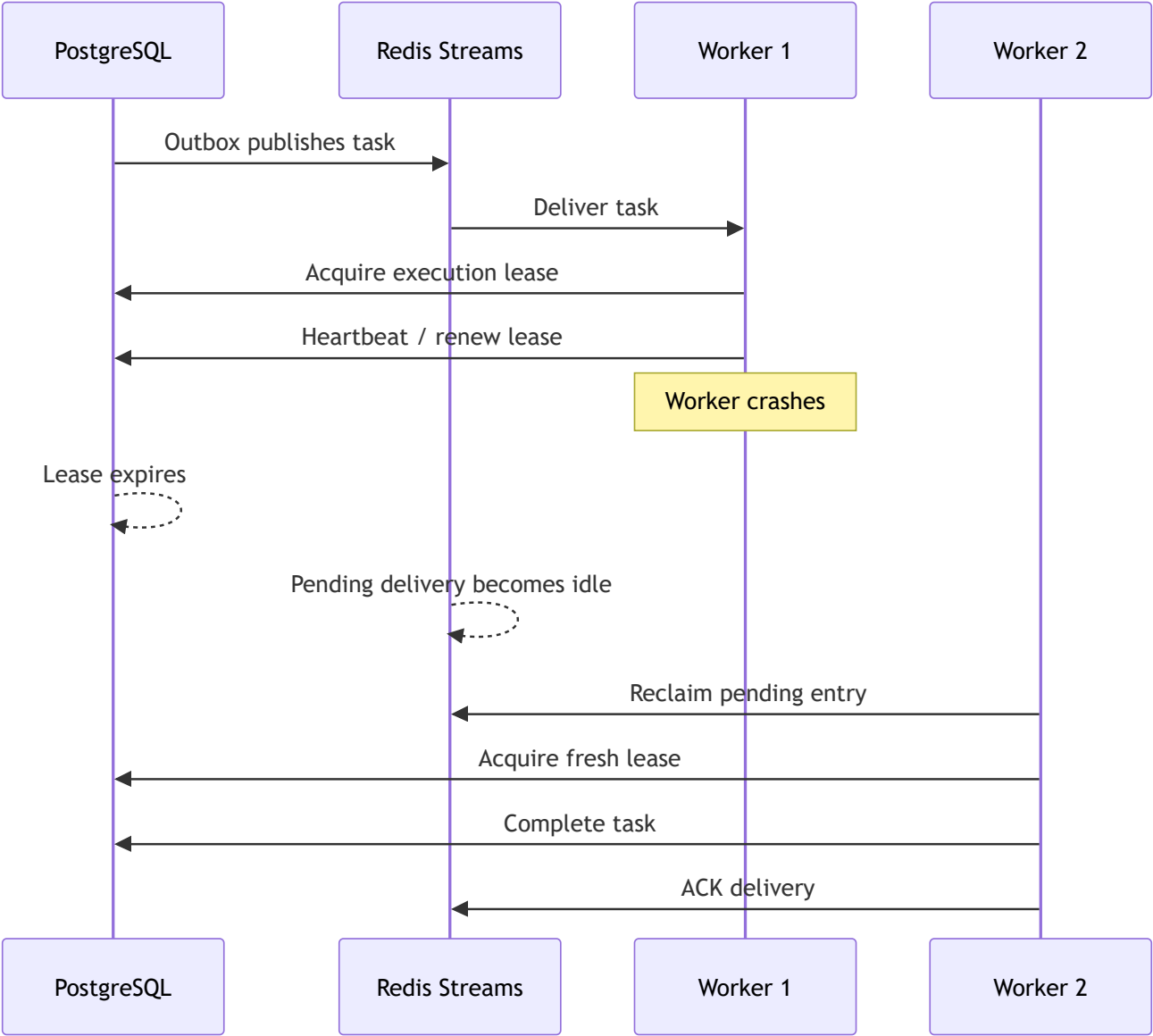
42.3 Capability-to-Protocol Execution Flow



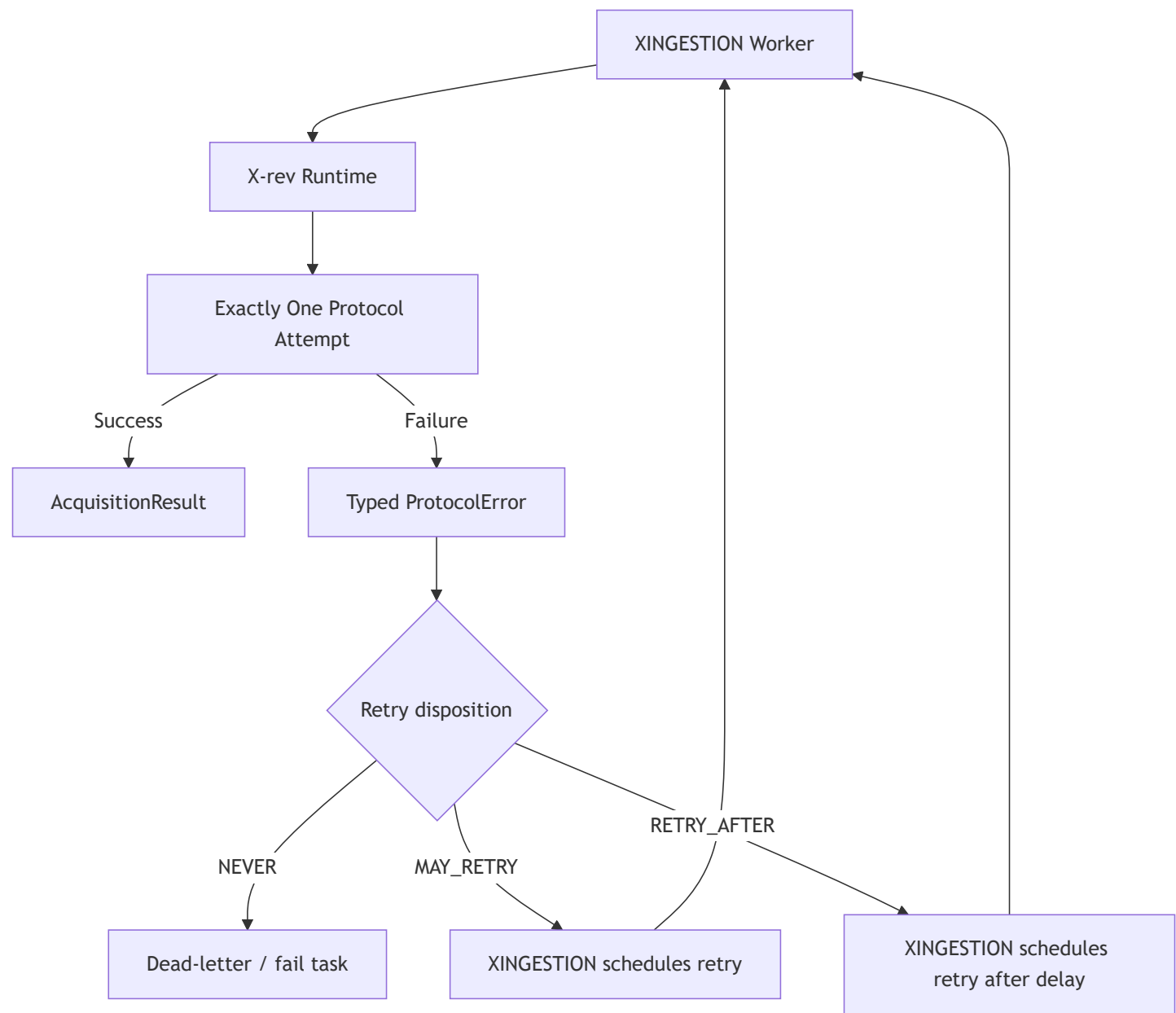
42.4 Production Task Lifecycle



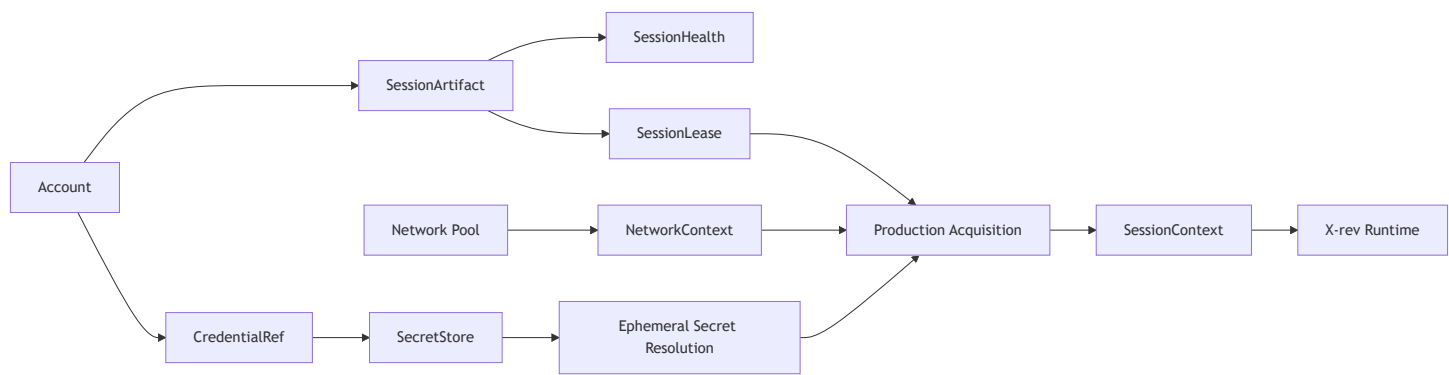
42.5 Worker Lease and Crash Recovery



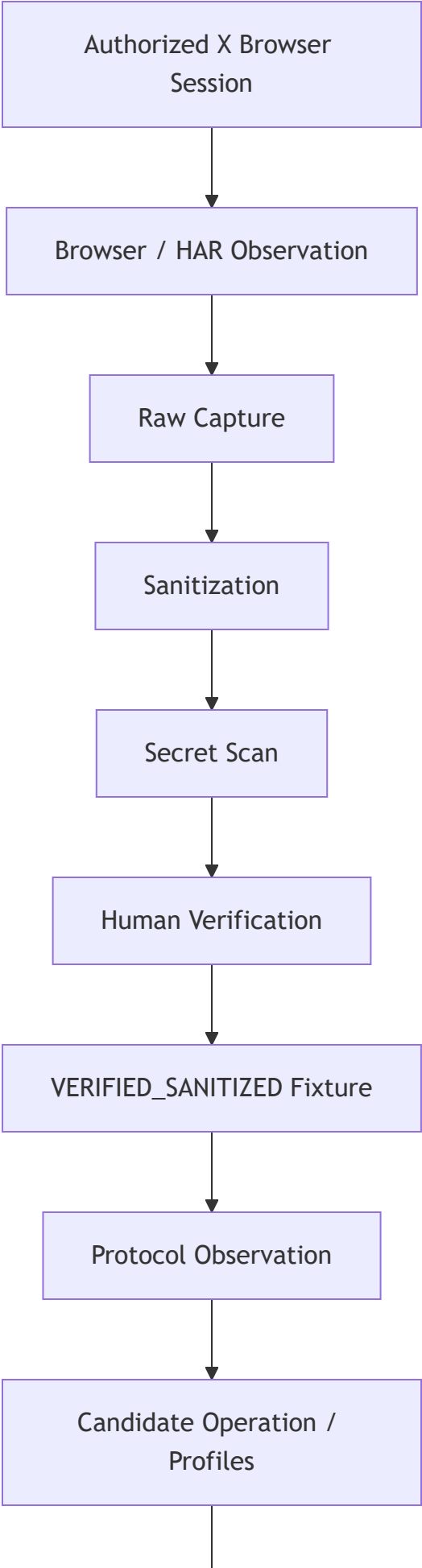
42.6 Retry Ownership

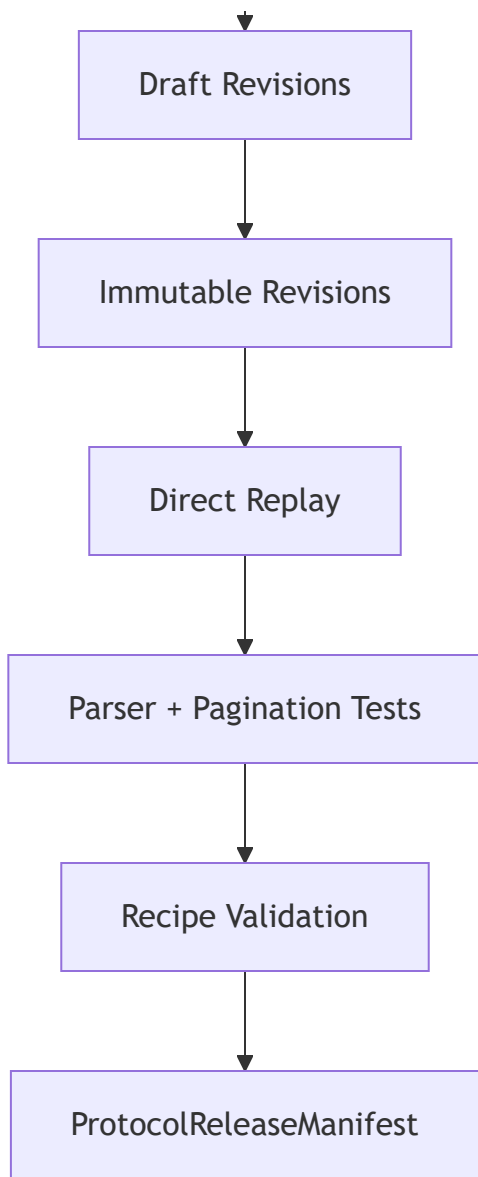


42.7 Session / Identity / Network Model

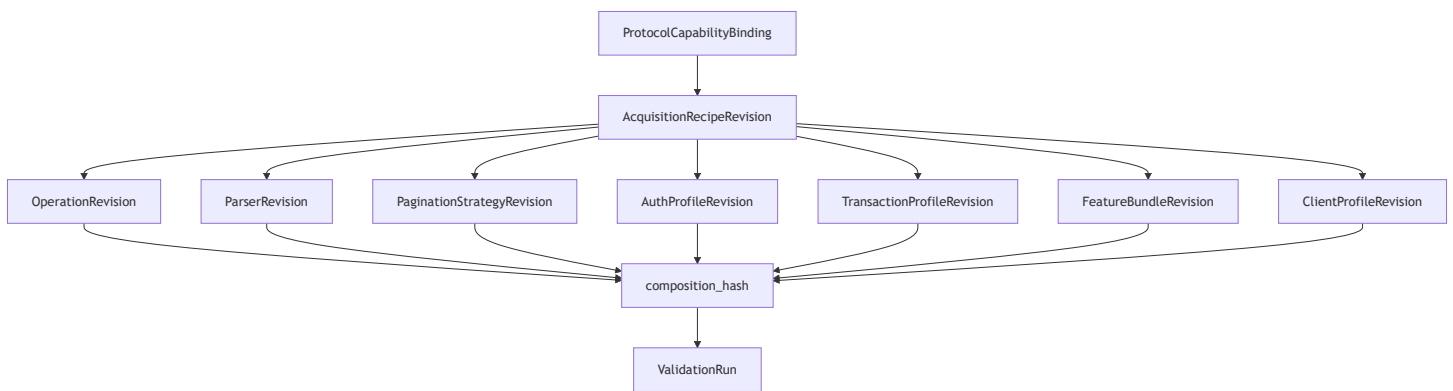


42.8 Protocol Research Lifecycle

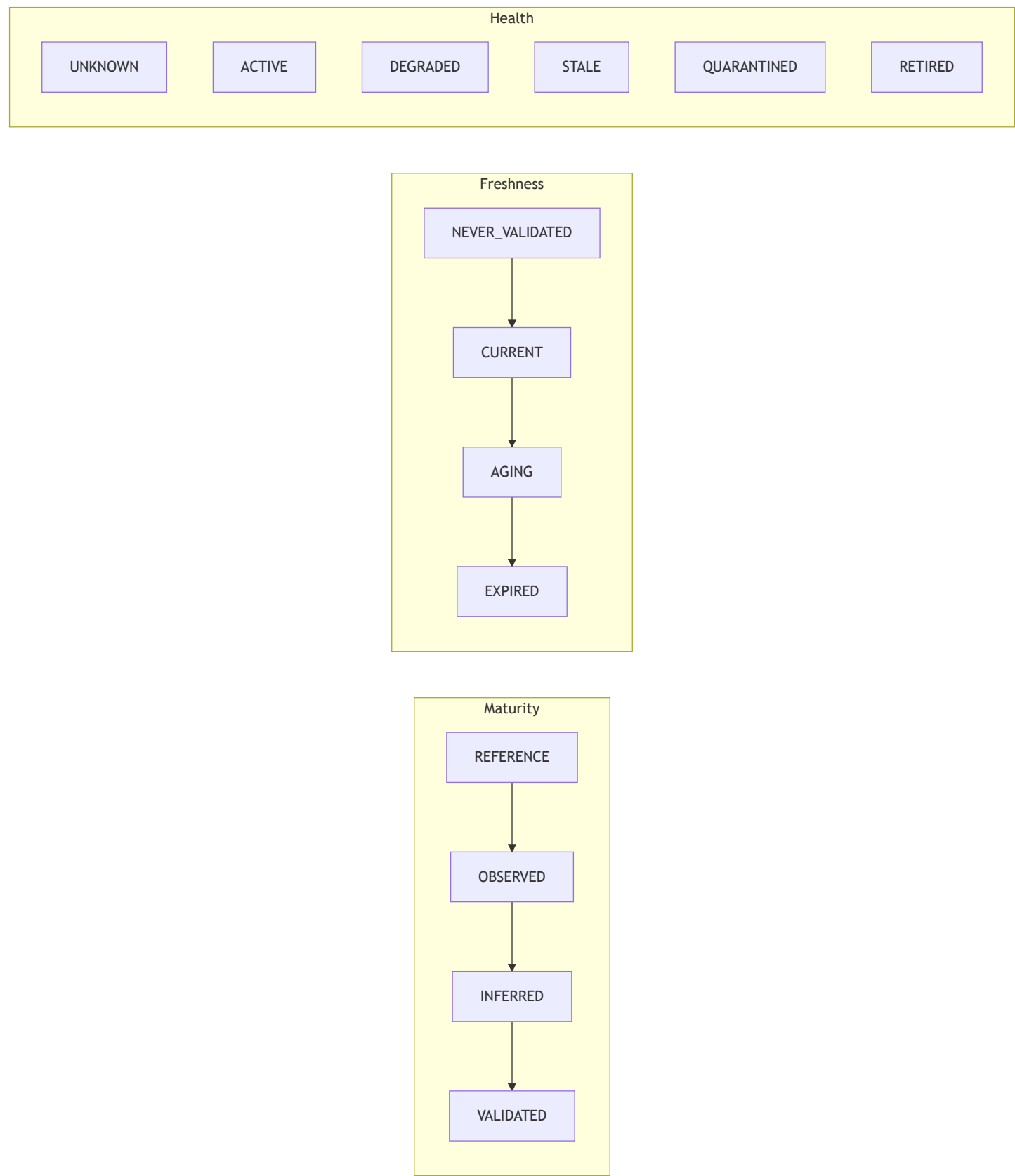




42.9 Exact Recipe Composition

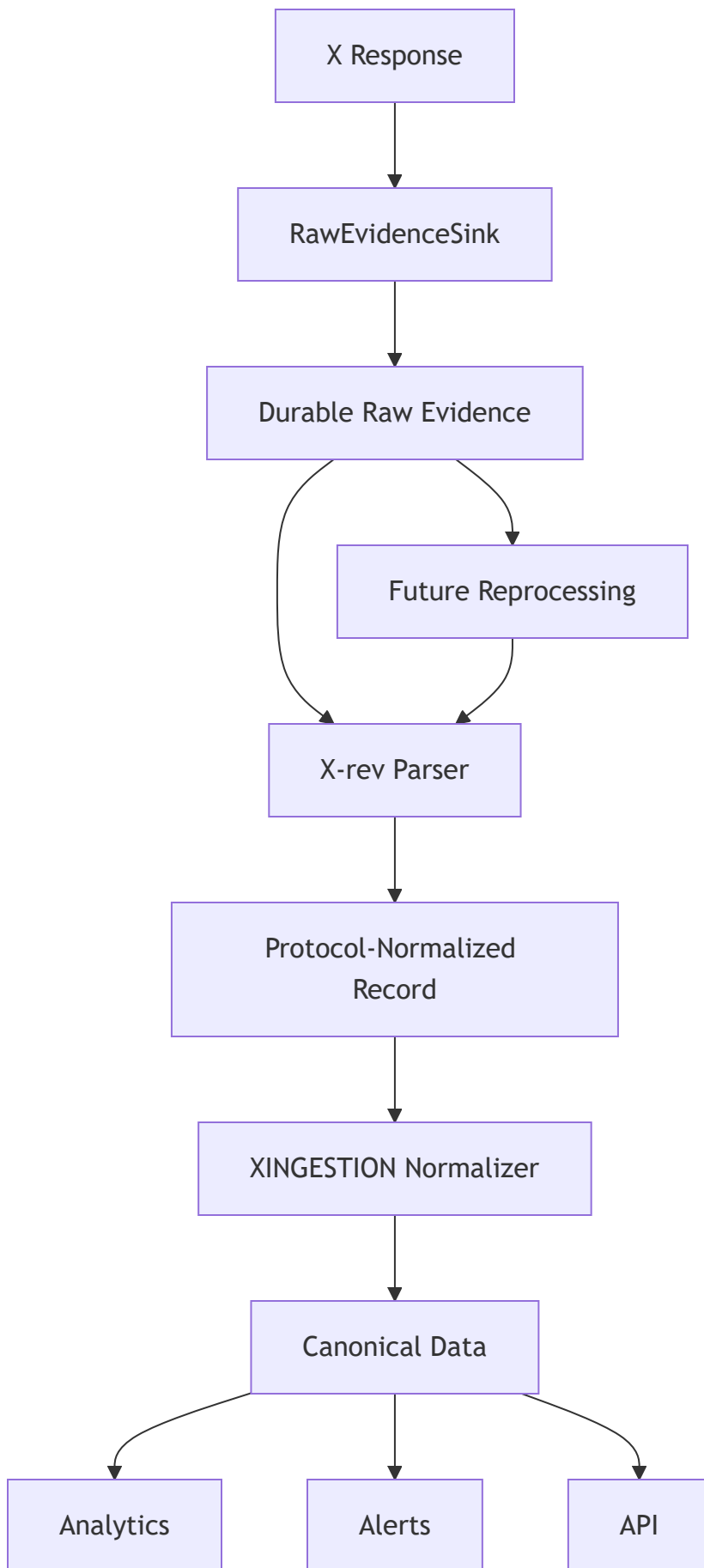


42.10 Validation State Model

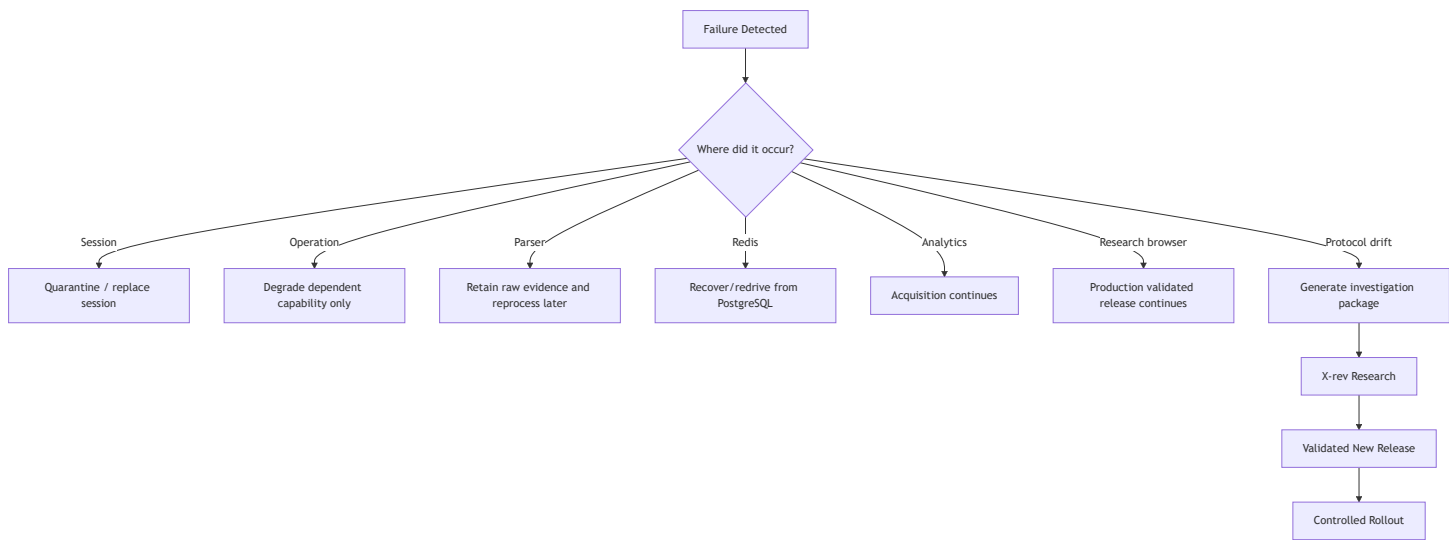


These three dimensions are intentionally independent. A recipe may be
VALIDATED + EXPIRED + ACTIVE .

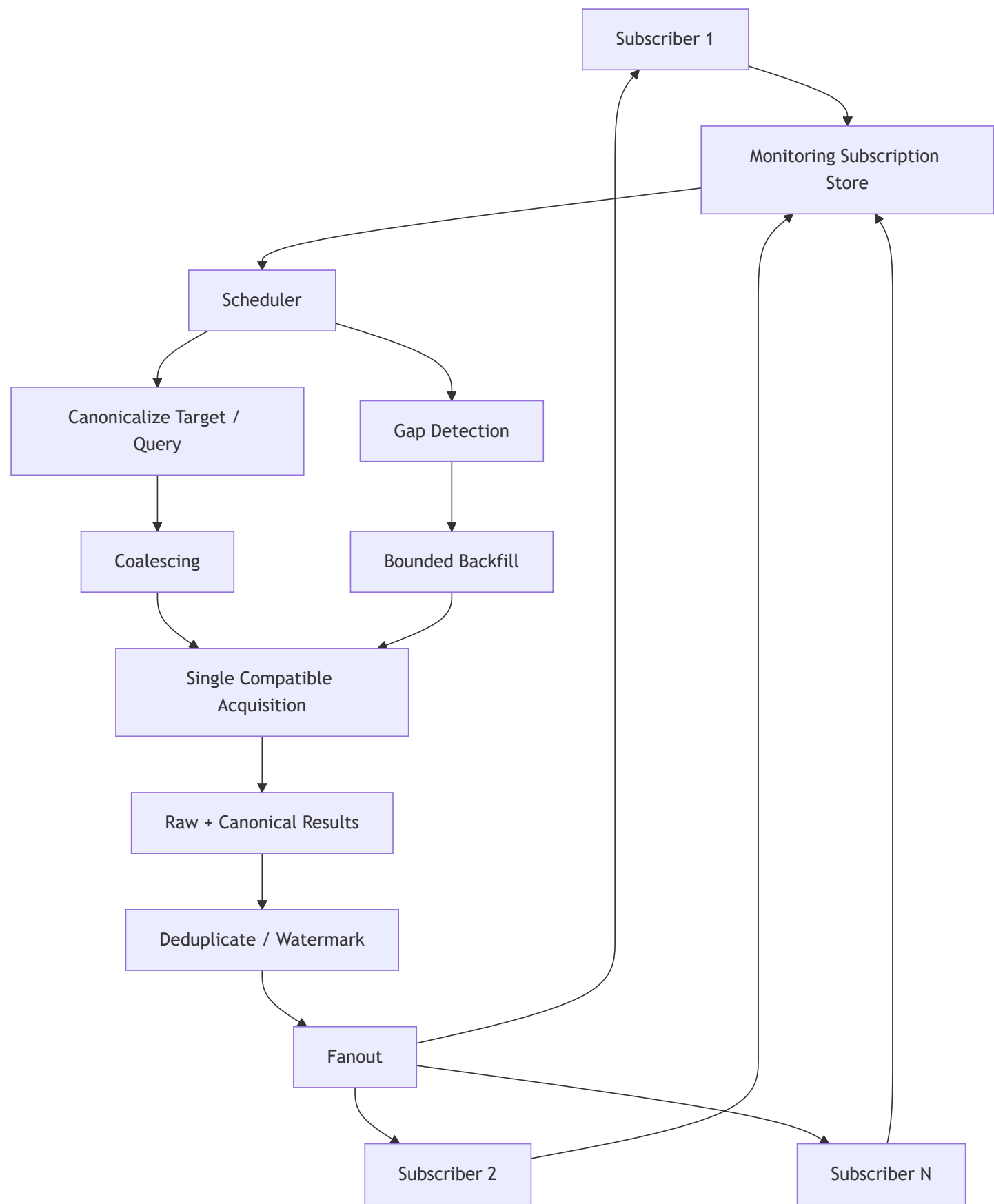
42.11 Raw Evidence and Reprocessing



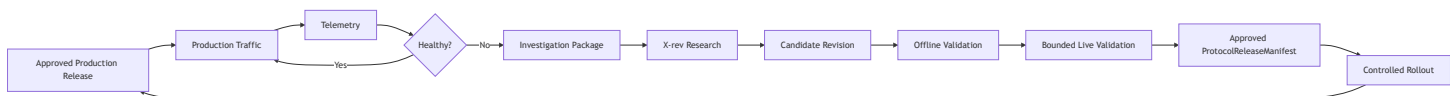
42.12 Failure Isolation



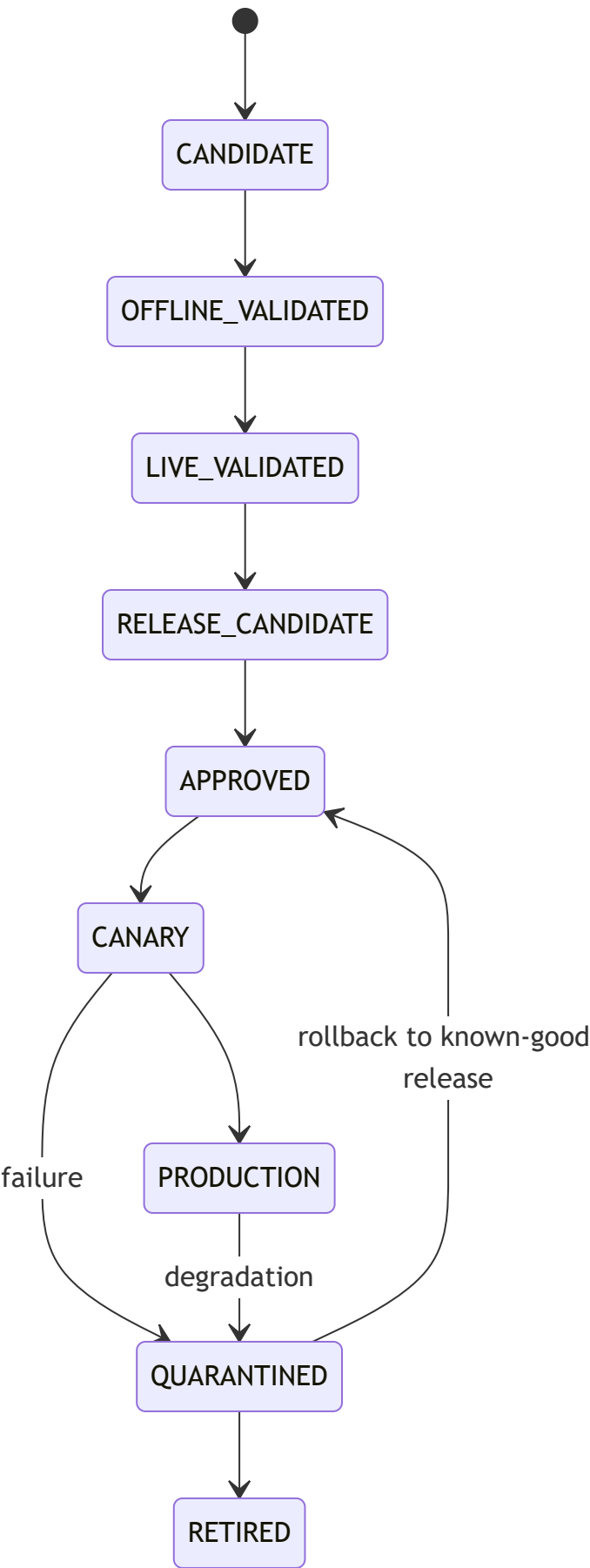
42.13 Monitoring / Subscription Architecture



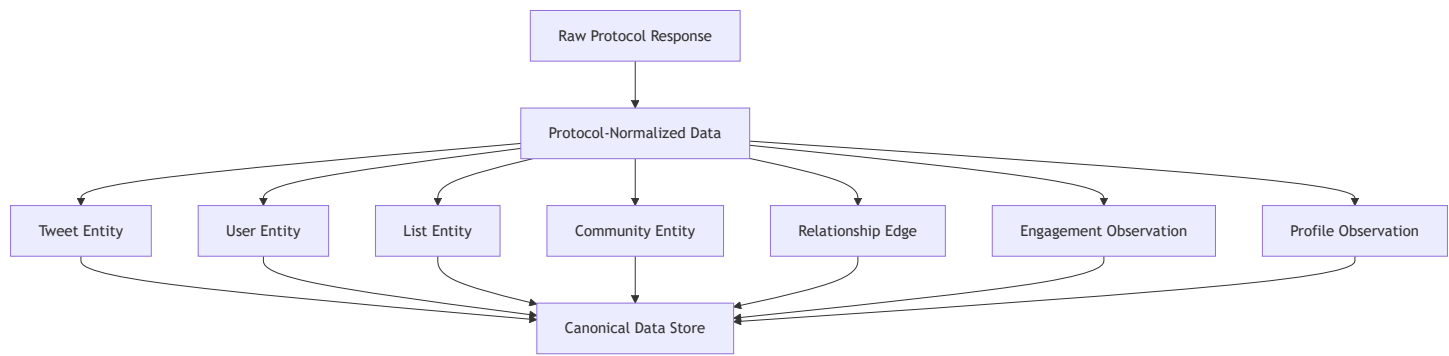
42.14 Protocol Drift Feedback Loop



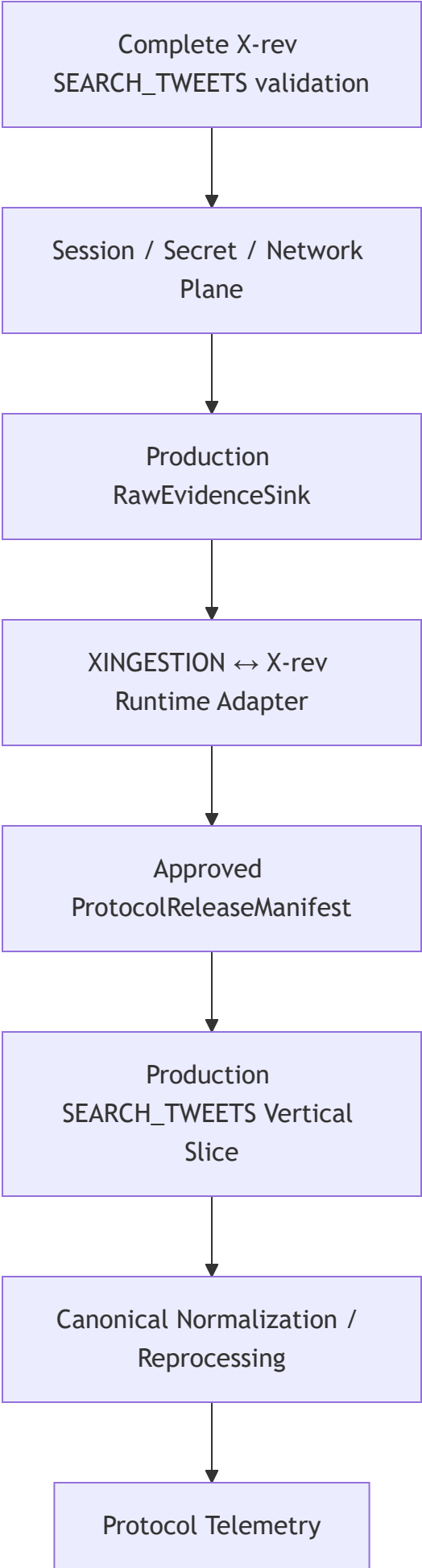
42.15 Release Promotion Lifecycle

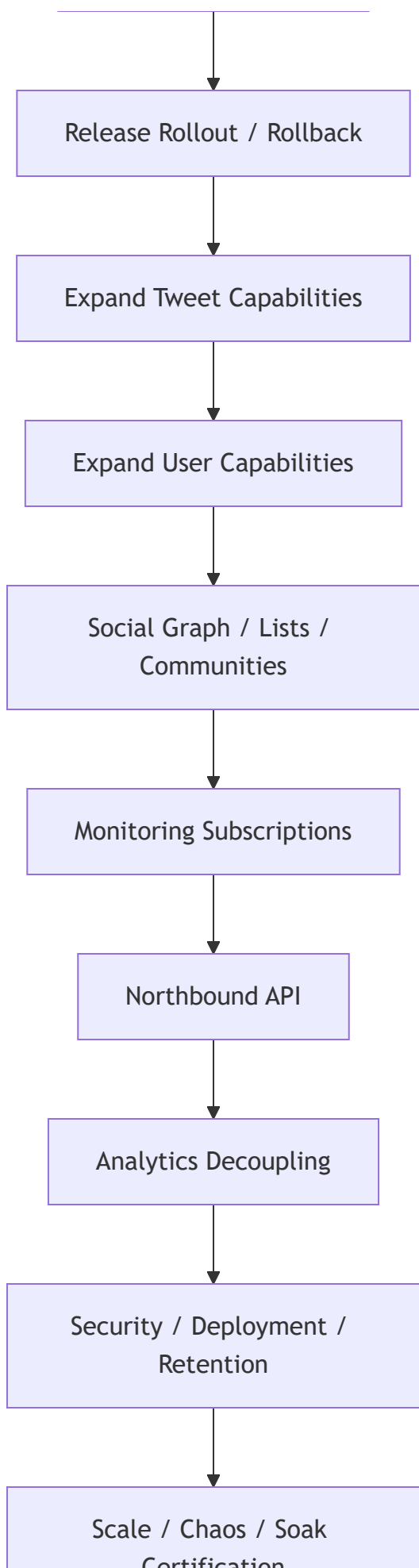


42.16 Data Model Separation



42.17 Final Implementation Roadmap





43. Mermaid Rendering Notes

These diagrams use standard Mermaid syntax supported by GitHub Markdown.

If a viewer does not render Mermaid blocks automatically, they can still be copied into:

- GitHub Markdown;
- Mermaid Live Editor;
- compatible Markdown editors;
- documentation systems with Mermaid support.

For repository documentation, these diagrams should remain source-controlled as text rather than exported only as static images so that architecture changes remain reviewable in Git diffs.